

软件仓库管理 (2)

生成式软件工程 · 南京大学 夸克原文图文转录

本稿保留夸克导出的原文、语序及段落，不做摘要或术语改写。原文含有语音识别错误，尚未逐句听校。时间标记来自 TXT 中的段落起点，不代表逐句字幕或精确结束时间。截图按时间归入最近的前置段落，图注保留原视频画面时间。

全片 01:38:29 | 173 个段落 | 108 张截图
来源：绿导师原谅你了 · BV1Q6en6NEUo

00:00:00

我们上周讲了一点版本管理，我们这周继续讲版本管理。然后我发布了一个作业，然后按照之前的说法，就是这一门课所有的作业大家都是选做自愿完成。然后最后当然肯定会验收，因为最终也许学校会让我们提交一个成绩，那么大家就会交一些链接，然后由我的机器人来给大家评分。



00:00:27

然后我们的第一个lab算是一个比较时髦的需求，叫会成长的个人助手。然后他现在对大家的要求都是不限，你们想怎么做就怎么做，然后最好还是可以web一个。而且我今天应该会客场会留的时间，我自己就来web code一个。然后我会告诉大家，如果你直接让他web的时候，你会遇到什么样的小坑。

00:00:52

然后除了今以外，我下一周而不是下一周是这周日要调休周二的课，所以这周日还有课。然后会讲软件的架构，然后会把它连起来。然后这个例子就是你们特别每个同学都会需要的，我自己自己也有，就是一个能够持续为你工作的个人助手。然后你可以这些需求是供你参考的。当然如果你实现它的话，会有一点点成就感，对吧？比如说你的学习的数据库，你你们这个学期可能会在上一些专业必修课，可能计算机网络或者概率这类比较困难的专业课。然后你可以把你的专业课的资料放到你的个人数据库里，或者你参与了研究组的课题的时候，你可以把你们研究组的论文相关的文献教科书这些整理成你的个人的数据库，这是一种形式的数据恐布线。你也可以把你自己的个人信息？



画面 00:01:00

00:01:45

我不知道大家是不是已经填表已经填的很烦了。每次你填表的时候都是给你一个DC的表格，然后在那个表格里要填姓名、学号、身份证号，然后有时候是个人简历。然后我现在已经把这些所有的事情都交给agent了，就好把这个东西填了。就觉得从以往到今年有一个非常大的效率进步。我今天我记得我大概在疫情的时候，我招过一个学生，然后研究生就在我这里做了三年，然后做了一个什么就是自动填表系统，我们就填觉得填表很烦。然后等到他毕业的时候，突然有了ChatGPT然后又很慌，他他做他觉得他做事情马上就要被替代了。我说不要紧，你赶快答辩，然后就可以毕业了。对，然后这世界就是发展的这么快。



画面 00:02:00

00:02:32

然后在个人数据库的基础上，你可以把数据库和你的s mail的邮箱关联起来。如果有也就有可能的话，比如说我在两周以后会在半夜给大家发邮件，然后你的agent的可能会把邮件回给我，就实现一个always on的一个在后台。你们也许买一个很小的那种嵌入式的小设备，一两百块钱你就可以在你的宿舍里面持续的p你的agent loop这样。然后以及用你的信息可以办理一号的事务，就像我上课的时候。对，这就是一句话，你直接一个像agent的紫禁城的调用就可以。然后如果你们有兴趣的话，可以做一个手机端的联动，也许是一个网页，也许是一个别的，甚至是一个苹果的app或者是一个android app。就你只要让a ent去做，他会自己去帮你下好所有需要的东西，然后把你要的东西给给给做出来。

实验 (1): 会成长的个人助手

本课程的实验是建议性质的实验，不记分。你可以直接将实验要求交给模型，配合一个 *Grill Me* 之类的技能，让它追问你的需求，再一起完成实现。怎样把“我想要”变成可以验收的软件，也是实验的一部分。

1. 背景

课堂上，我们已经展示了通过浏览器自动完成日常信息化流程的案例。有了 *Agent*，我们可以期待更多：让个人资料、邮件、办事大厅和手机连接起来，把一次成功的尝试变成随时可用的能力，并且在被你纠正之后，下次能把事情做对。

2. 实验要求

实验要求：实现会成长的个人助手

实现一个持续为你工作的个人助手，具备以下五项基础能力。可以基于现有 *Agent* 配置工具、脚本和技能：语言、模型和交互形式不限。

1. **个人数据库**。能索引、检索你的资料，并随资料变化更新索引，回答时能定位到原文。格式不限，推荐 *Markdown* + 目录结构：你可以直接阅读、修改文件，也更容易观察 *Agent* 读了什么、写了什么、以及如何组织记忆。

2. **持续关联 email 邮箱**。能持续获取新邮件、关联历史往来，结合个人资料拟好回复草稿。你可以编辑草稿，再直接发送；发送的应当是你确认的最终版本。重复检查邮箱不应重复处理或发送同一封邮件。

3. **办理 ehall 事务**。能在 *ehall.nju.edu.cn* 中查询信息，并完成至少一种事务的材料准备和表单填写。行为要保守：提交前展示关键字段及后果，由你明确确认；退课、撤销申请等操作尤其不能自行决定——万一 *Agent* 把你的课都退了，你就完蛋了。

4. **手机端联动**。能方便地从手机发起任务、查看进度、编辑草稿和确认操作。可以做 *App*、网页，也可以借助手机已有功能。例如，在 *Mac* 上用 *AppleScript* 操纵备忘录，再通过同步与手机联动。任务不能依赖桌面端某个始终开着的对话窗口。

5. **能力组合**。上述功能应当能共享必要的上下文，完成跨工具的任务。如何组合由你决定，但至少做出一条你愿意反复使用的完整流程。

2. 实验要求

实验要求：实现会成长的个人助手

实现一个持续为你工作的个人助手，具备以下五项基础能力。可以基于现有 *Agent* 配置工具、脚本和技能：语言、模型和交互形式不限。

1. **个人数据库**。能索引、检索你的资料，并随资料变化更新索引，回答时能定位到原文。格式不限，推荐 *Markdown* + 目录结构：你可以直接阅读、修改文件，也更容易观察 *Agent* 读了什么、写了什么、以及如何组织记忆。

2. **持续关联 email 邮箱**。能持续获取新邮件、关联历史往来，结合个人资料拟好回复草稿。你可以编辑草稿，再直接发送；发送的应当是你确认的最终版本。重复检查邮箱不应重复处理或发送同一封邮件。

3. **办理 ehall 事务**。能在 *ehall.nju.edu.cn* 中查询信息，并完成至少一种事务的材料准备和表单填写。行为要保守：提交前展示关键字段及后果，由你明确确认；退课、撤销申请等操作尤其不能自行决定——万一 *Agent* 把你的课都退了，你就完蛋了。

4. **手机端联动**。能方便地从手机发起任务、查看进度、编辑草稿和确认操作。可以做 *App*、网页，也可以借助手机已有功能。例如，在 *Mac* 上用 *AppleScript* 操纵备忘录，再通过同步与手机联动。任务不能依赖桌面端某个始终开着的对话窗口。

5. **能力组合**。上述功能应当能共享必要的上下文，完成跨工具的任务。如何组合由你决定，但至少做出一条你愿意反复使用的完整流程。



画面 00:03:00

00:03:26

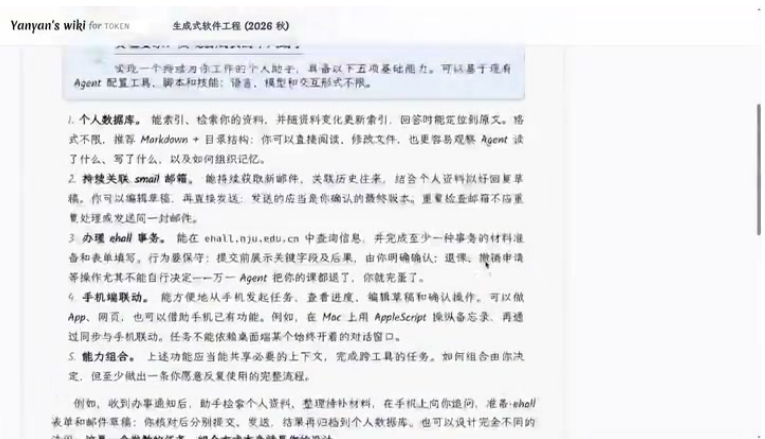
对嗯，然后为什么要布置这样的实验呢？就看起来就所有的任务都在AI的能力范围之内，对吧？但是如果你直接我今天会试的，我没有试过直接把这个实验要求直接让AI I去做。他在一开始就会做出很多可能，他认为看起来也合理的决定。但是这些决定会让你那项目的维护变得越来越困难，最后直到陷入角流坑。所以我觉得是每个同学都有有必要体验一下软工程里面的郊游坑。也就是你当你的a sla开始不断的消耗你的token的时候，你终于有一天会下定决心，好，我要把这个项目删掉，从头重做，或者就回退到从某一个很早期的版本开始重头做。



画面 00:03:39



画面 00:04:00



00:04:14

这是我们这个实验可能想要告诉大家的事情。好，当然我相信这个项目对每个同学来说都是有用的这是我也是我设计实验的初衷。我大概会这样，我可以提前预告一下，我的下一个lab会和这个lab有非常大的关联，而且你们暂时猜不到。然后，你会发现你就你现在la 1，比如说那些实验要求你都做好了，然后他成为一个非常舒适的个人助手。然后我怕给你加一个需求，然后这个需求和前面所有都是关联，并且有一定的冲突，然后你的意见就去改，然后越改越崩溃，就有一个这样的体验。所以大家可以期待一下这个后面的la所以如果你你不做，比如说你现在live一选择躺倒，或者就不管他让AI I直接就做，下一次再把这个做的直接删掉。



画面 00:05:00

Review & Comments

Lab1: 会成长的个人助手

- 一个非常“时髦”的需求
- 以能帮上自己为目标
 - | ✨ “成长”的验收：这次纠正，下次还能生效。

00:05:11

可能就没有这个软件工程的体验了，大概是我想要做这样的一个，好，我们回顾一下，我们上节课讲的是软件的项目管理？项目我们说项目是各种文件的集合，你的intent，一些设计文档。你的read me都在都在你的项目里。所以其实我不喜欢那种AI I写出来那个非常长的这个read me对吧？我喜欢的是对我来说，我觉得read me应该就写一下你在设计这个项目的时候的最初的初衷？你想做一个什么样的产品，然后这个产品能达到什么样的效果，可能配两个screen shots，以及你最重要的那些设计。有点什么感觉呢？有一天你突然想了一个非常好的点子，然后你真的想用它去挣一个亿。

00:05:58

然后你想挣一个亿的时候，你组建一个创业团队，你要说服你那个computer science里面？比如说你要说服我出来干，然后你就只其实只需要一个比较短的read me对吧？这是你的in能，你想做这个东西。那剩下比如说我有架构的能力，我有写代码能力的，我可以把剩下东西都给你补出来。



画面 00:06:00

项目是各种文件的集合

Intent、Spec、Code、...



2/4

00:06:17

你不需要知道这个软件系统是怎样，对吧？他的specification是什么？他用什么样的数据库，怎么样支持这个用户从1到10到100到100万到1亿？这个价格你可能没有commercial science的知识，但是你不需要有，你只要有个最原始的设计，那in然后再到spe就用我刚才说的一些我要选什么样的数据库，我甚至要选什么样的编程语言，然后我的项目的结构是什么等等各种各样的。小到比如说每个模块的接口都属于，然后再到code implementation，他现在都在这个项目里，那有这样的项目以后，很自然的你就就需要一个容器，把这些各种各样的文件给管理出来。

00:06:59

我们说大家用文件系统来管理项目，其实真的就是图方面。因为你回到1950年，1960年的时候，操作系统给你的存储功能就只有文件，它没有别的。就那个时候数据库还关系型数据库还没有发明出来。所以所有的程序员他别无他法，只能把你的项目以文件的形式存进来。所以这更多时候是一个历史的对吧？由历史不停的往后涨。



画面 00:07:00

文件装得下什么？

文件系统的方便

- 其实“文件”只是文件系统的方便
 - 其他形式(例如 db) 对人类和工具都不太友好
- 但丢失了很多文件不适合表达的信息，例如 artifact 之间的 *traceability*



10 分钟 → 30 分钟：哪些实现和测试需要复查？

3/4

00:07:27

导致的原因。我自己是觉得文件系统可能不是组织项目的正确的方式。因为文件系统其实丢失了很多文件不适合表达的信息，就是有些信息其实是比较容易表达的对吧？比如说我的intent spi这种，就我一条文档这个地方我是怎么想的，或者这个测试用例是什么，我输入是什么，输出是什么。然后我要sert在程序的运行过程中，什么东西不能被违反。但是你马上就会发现有很多隐藏的关联在项目里面是没有的。我举一个例子，你们你经常会在AI I就是AI I在写代码的时候，现在A认会稍微好一点，有时候也不会你让他说把这个实现的什么方法，从比如说你要把link list换成是一个更高效的结构去优化性能，对吧？这是一个很常见，你突然发现这个性能不够的，然后你让AI I去做AI I就***给你代码给改了。



画面 00:08:00

项目是各种文件的集合

Intent、Spec、Code、...



3/4

00:08:26

但是你没有想到的是，你可能在其中的某一个文件里写了一个文档里mark down里写了一个我这个项目是用这个数据是用linlist的管理的。然后在项目当中其实你的这个文档和你的代码和测试之间就形成了一个关联，对吧？它就形成了一个你在写的时候，你写文档的时候可能还没有代码，所以你就先写的那个文档，你随手一写，这是一个link list。但是一旦你的代码实现了这个关，又没有强行的联系上，这就是一个技术债 technical de对。因为以后随着文档和代码的不断的向前推演，他们的不一致只会越来越多，不会越来越少。



画面 00:09:00

文件装得下什么？

文件系统的方便

- 其实“文件”只是文件系统的方便
 - 其他形式 (例如 db) 对人类和工具都不太友好
- 但丢失了很多文件不适合表达的信息。例如 artifact 之间的 *traceability*



10 分钟 → 30 分钟：哪些实现和测试需要复查？

00:09:14

然后这个软件陷入一个维护的泥潭了。因为后面你拿出一个代码，甚至AI拿出看到一份代码，那到底是文档说的对还是代码说的对？你跟他聊天的那个历史没有了对吧？你那个chat已经消失了，你换了一台电脑对吧？然后你那个grile里面可能还有一些change的日志。但是你你当你到最后的时候，你真的就不知道到底是代码队还是文档，对吧？

00:09:41

然后其实在软件工程里面，这是一个研究的非常长时间的问题，叫traceability。你看这是这已经是2014年的时候，他review的一些更老的时候的文章，对吧？它就是suffer tracability可追踪性。所谓的追踪就是你的就他发现我们文件真的只是图保存在文件系统里方面。那么在一个软件项目当中的各种各样的测试用例文档，哪怕就文档中间一行代码，它之间的这个关联是什么？我们能不能在这个软件系统的演进过程中仍然维持它，对吧？这其实是一个非常软件工程的软件工程的问题。



画面 00:10:00

文件装得下什么？

文件系统的方便

- 其实“文件”只是文件系统的方便
 - 其他形式 (例如 db) 对人类和工具都不太友好
- 但丢失了很多文件不适合表达的信息。例如 artifact 之间的 *traceability*



10 分钟 → 30 分钟：哪些实现和测试需要复查？

00:10:28

所以很多时候我们觉得蛮有意思的，文件系统将错就错了。我们大家最后总有一天会发现，你用文件系统来管理代码，你受不了了，对吧？打个压缩包发布，打压缩包发布，然后打压缩包发给你的合作者，这件事情已经维护不下去了，对吧？维护不下去了，所以就有SVN有了git。那下一个是什么？大家有没有想过下一个是什么？我今天一会儿会讲它的下一个另外一个可能设计比git要好一些，更好用的一个版本控制工具。



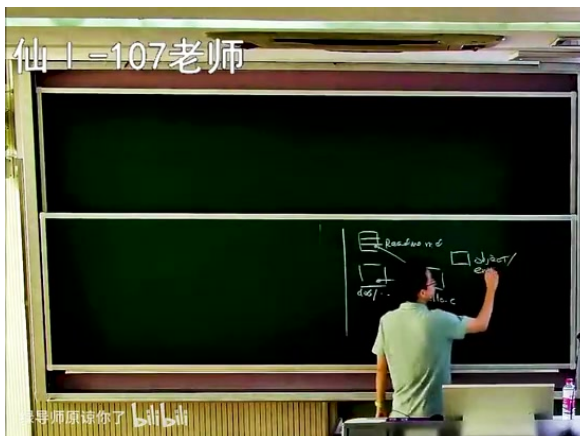
画面 00:11:00

00:11:05

但我同时也觉得，就刚才说到traceability，其实如果以后我们再也不需要写代码了，那软件它就不应该是一个就是像现在我们在git里面管理的样。我们好多文件，什么什么read me点MD然后什么hello点C，还有一个什么do下的什么文档。后其实如果如果所有的这些东西，我们人都不几乎不管了。那我们有没有可能我一直在想，就等这件未来到来，我们把这里面软件或者说部件之间，就也许我们还是需要文档，但我们的文档可能是好多个切片对吧？好多切片就把它们变成好多的对象object或者是entity。

00:12:00

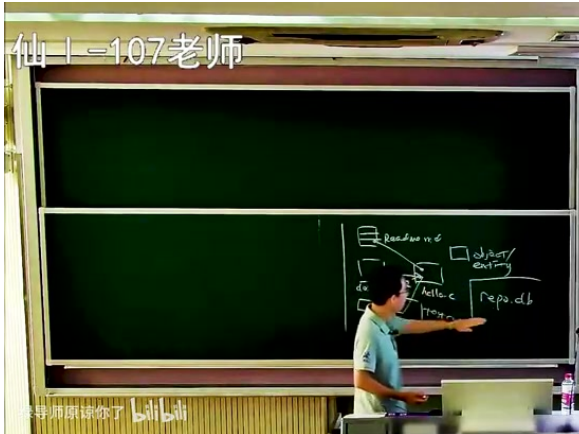
然后比如说这个文档和文这个实现之间就有一个关系，它实现了这个代码，实现了这个文档，对吧？然后我可能有一个测试用例，然后这个测试用例也有这个文档，那这个文档和测试用例又关联起来了。然后这个测试用例是和这个代码的这部分实现是关联起来的，然后这个文档又和这个文档是关联起来的。然后你看这是一个完全结构化的东西，没有办法在文件系统里一个竖账的形式表达，对吧？这是一个就随便连接的一个图，有cycle的图graph。那这个东西比如说我可以把它放到一个数据库里，对吧？叫这个就叫图数据库显然人类就没有办法用现在的这个unix的世界里面的工具去维护这样的一个repository了。



画面 00:12:00

00:12:48

但是不要紧，我们应该怎么或者说我们应该怎么解决呢？用魔法打败魔法，人类理解不了东西。我们只要训练AI就他只要有强化学习的反馈信号，我们就可以训练AI AI能够理解这个rec所以我们要干的是什么呢？



画面 00:13:00

00:13:03

这就是一个非常大的项目，什么Linux内核和数据库，或者你们想做的一个价值一个亿的产品。好，那每次你就说好AI因为你你你也不太可能说一下子就说我要把这个产品直接变成另外一个产品了。一当你比如说这个软件系统已经在工作了，那你一般来说就是要给它增加一个功能，修改一个功能，对吧？就跟get的philosophy很像，对吧？

00:13:27

Get管理了文件目录快照和快照之间的关系，你的get希望我们能够小步前进，对吧？我每做一件事情，我fix一个bug，我改一个文档，我都产生一个commit。然后这样我就可以在多个人之间协作，每个人可以自己产生自己的commit。然后我们可以用像rebase mer啊这个fast forward这些或者是普通的这种merge来创建一个可追踪的整个项目的修改历史。如果我们以后也不要文件，就也这种也许还可以用文件系统来管理，那我们想要的就是我们可以让AI自动维护所有的这个artifact软件，artifact之间的一致性，这个artiva的之间一致性就比如说代码和文档必须要一致。



画面 00:14:00

00:14:15

我文档说这是linlist，我的实现里面必须是linlist。然后我的这个test case测试用例，我我我文档里面说我要有abb c三个方面的测试，那我这个test case就必须要有abc它都是关联起来的。然后，你想从get里面我们知道我们希望小步前进，对吧？我们希望小步前进。

00:14:34

同样的当我们谈要做。

00:14:36

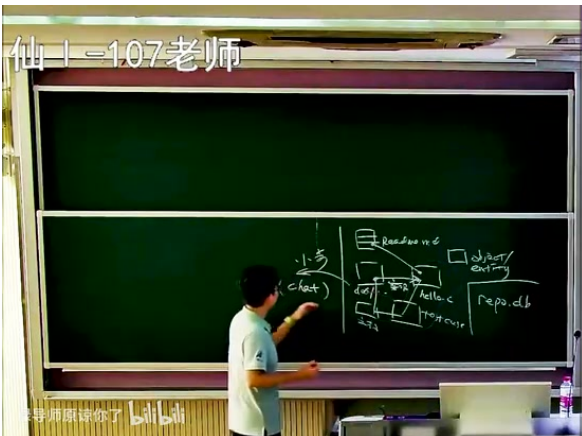
一小步的时候，其实我们知道这一小步是干什么的。所以你可以chat对吧？你可以跟AI chat说我现在要我现在发现它有个bug，或者我做了一个产品，比如说我现在今天对这个幻灯片样式不满意，那这就其实就是一个小步。后，这个时候一个大型的软件系统里面，绝大部分的内容跟这个小布都是没有关系的对吧？比如说我当我说我对这个幻灯片的样式不感兴趣的时候，我比如说大家可以看直接看我的看我这个ri对，看一下顶层的结构对吧？我有我有些assets，然后甚至还有这个点cache，有有缓存的系统，然后有这个P1、P2、P3、P4、P5，然后有一些脚本等等。



画面 00:15:00

00:15:22

任何一个软件系统，当你只做一小步的时候，你就只需要看到这个软件系统的一小部分，大部分东西应该是无关的。好，既然是无关的，我们就可以让AI根据我们的chat把和我相关的东西给摘出来，对吧？他可以知道，我现在需要的是这个文件，这个test case和这个文档，是其他的东西都不需要了。然后你想AI有什么能力？AI有动态生成任何内容的能力，everything is cold对吧？Content as code, code可以表达任何东西。所以AI可以根据我的chat生成一个和我要改的这一小步相关的。We有一个视图，在这里你看你们学到的个各种系统里面的概念还是有用的。



画面 00:16:00

00:16:12

就因为不是所有的同学都学过数据库，对吧？数据库里面有一个叫view的概念。他说我们我们的数据数据数据就在这儿了。但我可以创建一个不一样的查询，可以把它以另一个形式展示出来。对，就是这个很简单的想法，我希望把这些数据以另外一个形式给展现出来。但比如说当我看到的时候，我就看到的不是这些代码文档diff这些，而是一个可视化的一个看板。然后这个看板上大概说了列出了和我这个小布相关的一些东西，然后问我要改什么。然后我一把改完了以后，我同样的还是从一个从V0到V1。

00:16:52

它还。

00:16:52

是一个Rio点滴B但是这一小步就会把这些相关联的东西，比如说我要增加一种类型的测试用例，那它就会在这个测试用例里面，比如说我还是画一个这样的结构对吧？好，那他可能就会在这些地方加一些测试用例，这个地方加一些测试用例，这个地方加一些测试用例，然后有些地方可能要改掉，它就从一个一致的状态，就是各种错综复杂的关系，后做一次小步的修改到下一个版本。这样我们就好像软件工程就又变了，彻底变了。



画面 00:17:00

00:17:27

我觉得这是一个未来有可能发生的，就是在AI内部它是这样生产软件，然后从此以后我们也就再也不要看代码了。我觉得你们看到现代的这种现代ID比如说我我其实还蛮讨厌的，是他现在就打开默认就是agent mode，然后agent mo就是原来的那个，我们熟悉那个ID它就没有了。然后强迫你用个像chat类似于chat的方式去跟他沟通。但因为我还是相对来说比较精通古法编程的，有时候我还是对代码有一些掌控欲，可能是不好的，这个可能是不对的。但是未来可能会是这这可能是一个途径。然后你们当然我觉得同学们肯定也会焦虑的那如果未来这件事情真的有可能发生，而且就可能一两年以后大就就真的有一个公司把这东西做出来了，那我们学那么多东西还怎么办？对吧？



画面 00:18:00

00:18:21

然后其实你发现在我们思考，就在创造这些东西的过程当中，我们用的都是你们学习的这些经典及大成的 computer science 的课程。我刚才说的数据库里的视图？数据我用的借用了数据库的概念实体和关系，然后我借用了 get？我用 get 就这个模型没有变。Get 长期的开发经验说候，我们要一步一步向前。如果在这个模型底下我们是大步向前的那就惨了。对，当然也有可能发生就是模型面非常强，一个非常大型的项目，然就一步啪就变成下一个了。

00:18:56

这个怎么做？我们现在还不知道，但是至少在初期我们可能还是要沿用过去的大家积累的软件工程智慧去小步向前，然后通往那个极乐的世界。但我还是相信这个极乐世界有可能确实有可能是能。是能导的对所以这就这就引发了一个我觉得在大学里面，我觉得蛮有意思的一个评论。



画面 00:19:00

00:19:25

这个同学应该是已经毕业了，然后已经毕业。因为我上节课说我问那个同学们 get base，然后大家都不知道什么是 grebase。然后 he reach 了一个 valid point，他说学生时代大多数人是不和他人共同合作的，也就是做作业，所以 rebase 用的场景就很少正确。这个 point 是是是对的，就你们做作业的时候确实不会。然后我的回应是大概是这样的。但是如果你真的去往这个外面的世界去走一走的话，这个互联网开发者社区铺天盖地都是稳当。

00:20:00

所以你一定会在有一天你会留意过别人的项目，会看到位叫 contributing MD。然后一般来说这个肯你马上就问这个对吧？你们有没有同学问过？Tree 没关系，错了就错了。我没有对吧？我没有他。



画面 00:20:00

00:20:25

仓库根目录下的一份贡献者指南，通常回答我怎么参与这个项目。然后你看它里面就有这些，就项目的总体的结构，然后分支提交信息PR的规范和流程。那你如果看过就如果你你看过很多的项目的computer m你比如说你就随手打开看一眼，然后如果你发现里面东西都知道了，好，那你就知道了。如果你不知道，比如说有一天你一定会看到一个项目里面有一个说说我们你必须要在你自己的分支上rebase，你禁止你你这种我要保持线性历史，禁止你直接创建那种分叉的mer你，就会知道rebase是是干什么的。然后他它现在是什么呢？

同学产生的疑问

从信任老师到自己的理解

关于git rebase 0/10我想替老师的学生投个票。学生时代大多数人是不合和他人共同合作的。大多数学生应该都是单分支开发了，用rebase merge这些场景就少很多

2020-09-08 19:20 · 28 · 0 · 0

👤 清华的清华了 你这样理解：互联网，开发社区铺天盖地都是文档，如果你需要过别人的项目，看到CONTRIBUTING.md，这些东西一定会出现在你的视野里，并且很容易看懂。如果你不懂，就说明你压根就没过去看外面的世界，没有把各种东西弄明白的原始动机。从而已经开始和这个信息过载时代“被淘汰”建立相关性了。很长时间大家都是在觉得反正所有人都要，我只要比别人不笨一点就能赢这个游戏了，但AI时代是重新定义以下全部价值为零，当然另一说，我觉得我还早也被淘汰，但在那之前还是得学。

- 典型的“学生思维”(信任老师)
 - 老师不教的就是我不需要会的
 - 考试不考的就是我不需要学的
- 学习的目的：形成自己的理解 (first principles)

9/18



画面 00:21:00

00:21:09

现在我们的社会变得太快了，所以有一种学生自味其实不是批评批评大家。就是因为你们上高中的时候就是这样的，在高中的时候老师不教的就是你不需要会的对吧？考试不考的就是你不用学的。非常明显的的一个你们为了升学的利益最大化，这个是正确的。没有没有错。然后在你们到大学以后，你们如果还带着这个思维的话，你你你就就会会有这样的对吧？这是这样的gap，就是你一种新人类和旧人类之间的gap。

00:21:41

当然我相信每一个同学也还是可以成为成为新人类的。然后你只需要去有一些注意力，去注意到更多的东西。因为现在我有一个危机，就是AI时代的斩杀线以下的价值就全部归零。当然我觉得现在展现可能还展不到个各位985同大学这个conscience专业的同学。但是可能比如说那些低级的那种低级智力劳动，对吧？比如说会计从业者，好像我看过一个报道说中国可能是不是有1000万还是有多少会计从业者。那那这个行业里面顶端的人肯定暂时还是没有威胁的。但是在底下的大量的人都都面临被agent的替代斩杀的危险。

9/18



画面 00:22:00

00:22:27

所以你们需要在这些经典的课程上，不管是什么操作系统、编译器这些，然后还是软件工程对吧？这里面有些软件工程的智慧，你们学到一些经典的东西，然后最终形成你们自己的自己的理解。对，这是一些评论和复习get。

00:22:47

好，我们继续回到软件工程。不管是这个模型，就是我说我畅想一下未来会发生什么，还是我们就是get对吧？Get get他管理的是快照，也就是一个目录对吧？整个一个可能是也就是这样的一个东西的一个快照。好，我可以在快照上面做一个开发对吧？做一个开发他要求小布，我现在有一个两个开发者，一个是A一个是B我们就可以独立的在这两个上面做做开发。然后好，那这个东西是commit。

同学产生的疑问

从信任老师到自己的理解

关于git rebase 0/10我想替老师的学生致歉下，学生时代大多数人是不合和他人共同合作的，大多数学生应该都是单打独斗了，用rebase merge这些场景就少很多

2020-09-08 19:20 · 28 · 0 回复

👤 陈明德同学发了 回复 你这样理解：互联网，开发社区铺天盖地都是文档，如果你的目标过别人的项目，看到CONTRIBUTING.md，这些东西一定会出现在你的视野里，并且也会有开源，如果你不懂，说明你的知识还没进步到看开源的世界，没有把各种东西弄明白的源代码。从已经开和这个信息过载时代“被淘汰”建立相关性了。很长时间大家都是在发现正所有人都爱，我只要比别人不笨一点就能赢得这个游戏了，但A时代是看谁线以下全部价值为零，当然另一说，我觉得我还早也被淘汰，但在那之前还是得学。

- 典型的“学生思维”(信任老师)
 - 老师不教的就是我不需要会的
 - 考试不考的就是我不学用的
- 学习的目的：形成自己的理解 (first principles)

10/8



画面 00:23:00

00:23:34

然后如果你要做这个时候，如果你要想把它合起来，那在get it里面，在get的这个原始的模型里面，它rebase相对来说是不那么提倡使用的。它是说好你会创建一个两路的merge，它有两个parent继承了这两个的修改。但是这又带来一些问题，比如说这个merge完，比如说merge是由aa来A来干的那这个merge里面的一些内容，就由必写的这些内容它就变它作者变成A了。它其实有一个隐藏的作者修改对吧？比如说B在这儿加了一行，这个可能还能还能追溯到B就如果发生冲突。比如说发生冲突的时候，在resolve的时候，那冲突完了以后，这个甚就是由这是由A完成的，就是A完成的那这个就B改的代码，最后ownership又归到了A所以这会带来一些相对来说有一点小小的小麻烦。然后rebase解决了这个问题rebase。我还是横过来画这个，是不是第一排的同学有点看不到。

如果想重新设计版本管理系统？

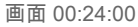
Git 经常需要处理 commit 之间的 delta (change)

- Cherry-pick, rebase, ...
- 但这是用 commit 来替代的 (感觉到轻微的不和谐了?)

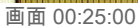
我们究竟想操作什么？

- “把这次修改搬到另一个版本上”——我们关心的是 **change**。
- 这次修改换了一个起点，它还是不是“同一次修改”？

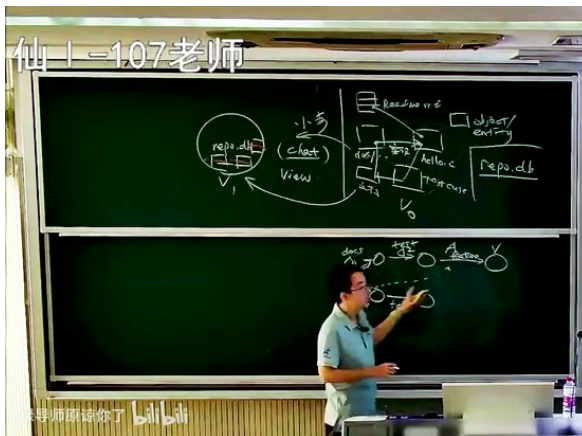
10/8



如果我现在有两个历史，那比如说我现在在我我我我我改了，这是A同学改了两份，做了两次的提交，得到两个两快找ok是他一他好，然后我这个时候另外一个同学开始改。好，那rebase做了一件什么事呢？Rebase说的是比如说A先改了文档，然后再加了测试，然后我B加了一个feature，然后又加了一个测试。那这个时候我不是直接把他们两个东西串合成一个说把这两条线路并起来的提交，而是我试图先把这个feature能不能在，我先要试一试有没有可能在这个A的后面把这个feature加上。如果加上了，ok那就成功了，这就成功了。



好，但是这是有一个很大的风险的，就是因为我可能在这个里面加入了一个测试用例，然后这个测试用例和这个feature就是不兼容的对吧？这是有可能的，或者甚至就是他们就改了同一个地方，我就做两个完全不兼容的东西，那这个时候它是有可能失败的。如果成功这就比较好，对吧？那我再把test放过来，如果成功，那么它就一路相当于如果就也就是说这两条线上那面的修改完全没有任何冲突，那我就可以一路把它合并过去，变成一个线性的历史。但是一旦发生冲突，这是非常危险了，为什么呢？一旦冲突你就需要resolve对吧？你会进入一个rebase的模式。你们如果如果进入过这个模式，你会发现瞬间自己就不知道应该怎么办了。



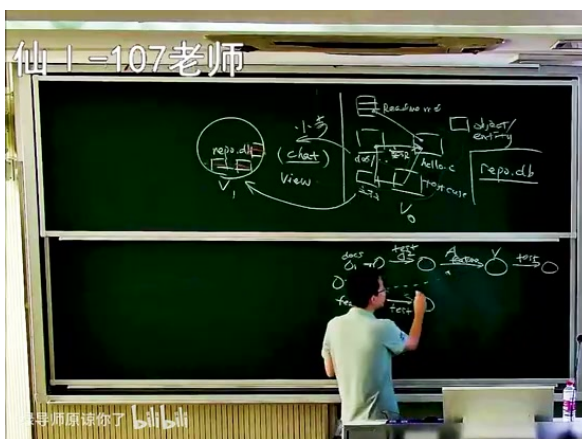
画面 00:26:00

00:26:44

然后这时候你就打开你的agent说，我现在在我先我我我发生了什么对吧？然后agent会细心的解释解释给你。更麻烦的是你就算是把这个东西搬过来了，feature搬过来了，你可能搬test的时候又要失败。

00:27:00

然后你还需要。



画面 00:27:00

00:27:02

再把test这个再搬过来。而且每一次你都会创建一个新的提交。也就是说这是一个commit，它有一个ID d这个commit有ID d每个comm都有ID那如果你是一个这样的两路的merge，那所有的这些ID都保留了，你会创建一个新的ID但是因为你是rebase的rebase，每次这样的rebase就会创建一个新的commit。因为他这个快照，本来这个快照的base在这儿对吧，这个commit的base在这儿，所谓的rebase你就知道rebase rebase它的base要改掉，要把这个base改到这里来rebase，那你会创建一个新的ID。每次都会创建一个新的ID然后这个历史就变了。然后你会感觉到这里有一点点轻微的不一致，对吧？就觉得你好像不想不是完全要想想这样去去管理软件。因为我们关心的是修改，也就是我希望的是把这个修改搬过来，然后在get it里面修改是用commit来替代的对吧？

如果想重新设计版本管理系统？

Git 经常需要处理 commit 之间的 delta (change)

- Cherry-pick, rebase, ...
- 但这是用 commit 来替代的 (感觉到轻微的不和谐了?)

我们究竟想操作什么？

- “把这次修改搬到另一个版本上”——我们关心的是 **change**。
- 这次修改换了一个起点，它是不是“同一次修改”？

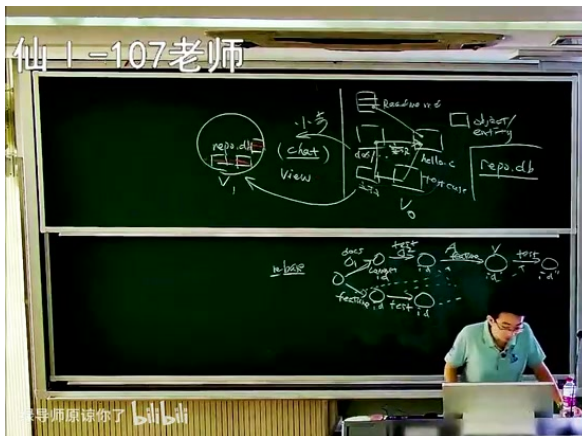
如果想重新设计版本管理系统？

Git 经常需要处理 commit 之间的 delta (change)

- Cherry-pick, rebase, ...
- 但这是用 commit 来替代的 (感觉到轻微的不和谐了?)

我们究竟想操作什么？

- “把这次修改搬到另一个版本上”——我们关心的是 **change**。
- 这次修改换了一个起点，它是不是“同一次修改”？



画面 00:28:00

00:28:20

说好我现在有一个commit，然后这个commit和它的parent之间的关系就叫一个修改。它没有一个专门的叫change的对象，所以它它是用commit来代替修改，那可能绝大部分的情况都还能满足。但是如果我们把change也变成1st class citizen，会发生什么呢？你就发明了下一代更好用的版本控制系统，对吧？就是说你你我我不仅可以有有committed的ID d甚至这个机器组，它是和get底层完全兼容的，你你你可以直接把它当get用。然后但是它增加了增加了一个叫做change的概念。也就是每一个这个东西。

如果想重新设计版本管理系统？

Git 经常需要处理 commit 之间的 delta (change)

- Cherry-pick, rebase, ...
- 但这是用 commit 来替代的 (感觉到轻微的不和谐了?)

我们究竟想操作什么？

- “把这次修改搬到另一个版本上”——我们关心的是 **change**。
- 这次修改换了一个起点，它是不是“同一次修改”？



画面 00:29:00

00:29:14

每个这样的东。

00:29:16

西change它都可以算一个一个change。然后它可以把好多的change，比如rebase的时候，它会把change直接把change搬过来，然后趁至还是有可能就冲突是解决不了的。如果我在rebase的时候，我要我要冲突，这是解决不了的。但是因为它有这个change的object，就change这个概念，所以它可以把这些change一度带下来，生成一个直接rebase到这里。但是里面引入了好多冲突，就你它还是应用了get的那个冲突冲突那个记录。但你看它的里面有一套自己的记录方式，把所有的这些change的冲突也都塞进去。然后你可以在这里相当于把它们搬过来，然后在这里再resolve它的底层的模型没有变，但是你用起来会舒服很多。

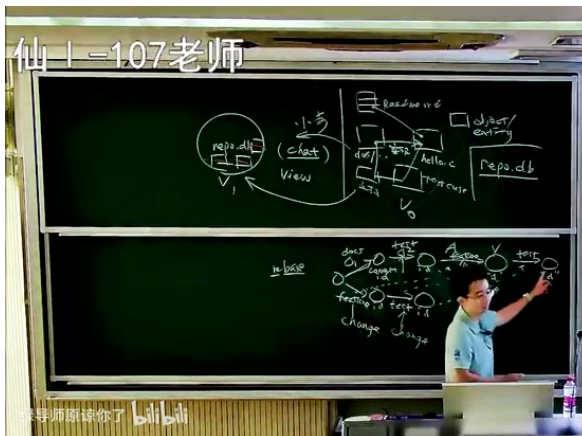
把 Change 作为 First-Class Citizen

祝贺，你发明了 **Jujutsu—a version control system**

- Rebase 不应该让同一次 change 失去身份。
- **Commit ID 会变，change ID 延续**：同一次修改，可以有不同的修订版本。[设计说明](#)



2/4



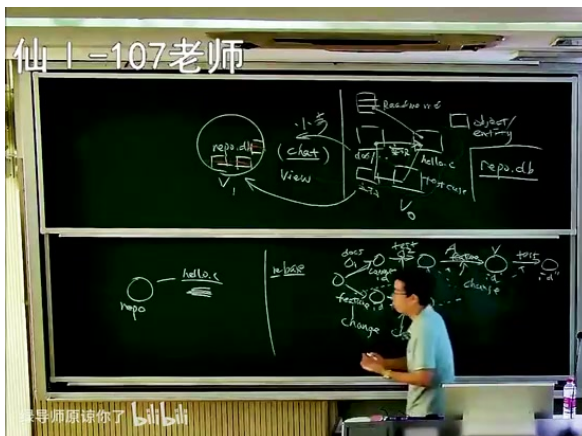
画面 00:30:00

00:30:06

然后它有一个很好的特点，它不需要staging area，就是你你你在你在get里面最讨厌的是什么呢？我上操作系统课的时候会讲这个例子，最讨厌是你现在有个工作区，对吧？你你你有个你现在有个repo然后你现在改了，比如说你有一个hello点C然后你现在把它改了改改改改改到一半，然后这个时候项目还不能编译，也不能运行，测试也通不过。你改到一半，突然导师给你打电话，我现在有一个很很高很高优先级的项目，你就有个bug，马上客户这个项目就要交不了差了，赶快把这个bug给我一分钟修了。好，那你怎么办？你不应该commit对吧？

00:30:57

是你需要你需要的是你你需要把它加到你的这个index里，然后get ash，然后它是会压，然后这其实是一个比较讨厌的操作，因为你是一个东西对吧？是一个东西，然后slash一个东西。你去把原来那个bug开始修，你刚修了一半，你的导师又给你打电话，你这个bug别修了，我还有一个更优先级更高的bug，你去把它修了。好，你这要再把它是然后get是一个站，然后它里面就会有访问好多好多多个提交。然后你你你去你需要去管理那个那个stack。



画面 00:31:00

把 Change 作为 First-Class Citizen

祝贺，你发明了 **Jujutsu—a version control system**

- Rebase 不应该让同一次 change 失去身份。
- **Commit ID 会变, change ID 延续**: 同一次修改, 可以有不同的修订版本。[设计说明](#)



让未完成的工作也有位置

连 **staging area** 都不需要了

- 仿佛一直在 `commit + amend`。
- **工作副本本身就是一个 commit**: 多数 `jj` 命令会自动记录文件修改, 更新这个 commit。[工作副本](#)

冲突直接存进 **commit (killer feature!)**

- Rebase 可以直接完成, 之后再统一处理冲突。
- **保存的是冲突的逻辑表示**: 带着冲突的 commit 仍能继续 rebase、merge。[冲突管理](#)
- **设计启发**: 把“尚未解决”表达成一种可保存、可继续操作的状态。

00:31:33

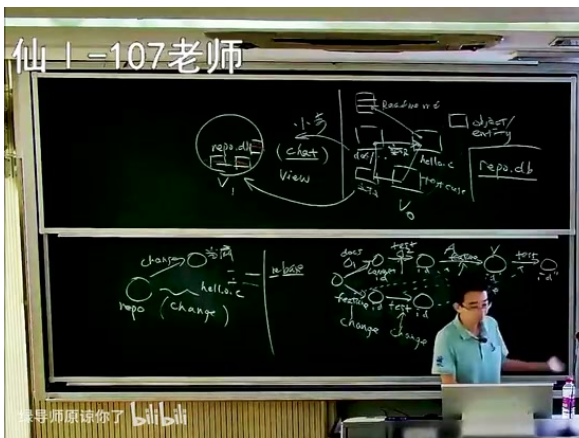
然后如果你想象想象如如果你想象在任何时候，也就是你随时随地都在编辑这个Repo，这不就是一个change吗？对吧？这就是一个change，你现在把hello点C改一改，加一行减一行，它都是change。所以你你所以这个矩句词它它好像是在干一件什么事呢？好像是你你但凡改了一个字节，它都会立即做一个get commit。当然这个commit不会产生好多线性的历史，而是先把上一次的那个就上一次那commit给抹掉。然后他上一次的commit就你改了一个字节，那上一次的那个提交就被抹掉了。抹掉然后你重新用当前这个修改去替代这个提交。也就是说你永远是把因为现在有个change，这现在有个当前的snapshot，它永远是提处于一个提交的状态，一个change提交的状态，那你就很舒服了，你连这个setting area也不需要了，这就是一个非常好的，它几个你从你从我要把change作为1st star class citizen开始，你再往后推，你就可以发明自己的版本管理系统。



画面 00:32:00

00:32:52

当然这个不是它的全部，然后它还有一些别的有别的路线可以推导出来。但是你们适当的时候可以质疑一下，比如说get就get有没有哪里做的不够好，对吧？Get哪里做的不够好，然后如果如果可以做的更好的话，还能做怎么样的设计？所以我觉得这是很有趣的一些非常有趣的设计。你们可以在agent的带领下，或者说我开了一个头对吧？我说我给你们一些motivation，你们可以把这些最重要的prompt交给agent，说我如果要学句句组，然后我想我想我想从change开始理解对吧？然后如果你看他的tutori啊，它其实也是一上来就会告诉你，它没有staging area等等这样的。你你管你管理这个版本是更自然，它对人类首先对人类更友好，然后实际上他可能会对agent也稍微更友好一点友好一点对，友好一点。



画面 00:33:00

让未完成的工作也有位置

连 staging area 都不需要了

- 仿佛一直在 commit + amend。
 - ✦ 工作副本本身就是一个 commit；多数 jj 命令会自动记录文件修改，更新这个 commit。[工作副本](#)

冲突直接存进 commit (killer feature!)

- Rebase 可以直接完成，之后再统一处理冲突。
- ✦ 保存的是冲突的逻辑表示：带着冲突的 commit 仍能继续 rebase、merge。[冲突管理](#)
- ✦ 设计启发：把“尚未解决”表达成一种可保存、可继续操作的状态。

让未完成的工作也有位置

连 staging area 都不需要了

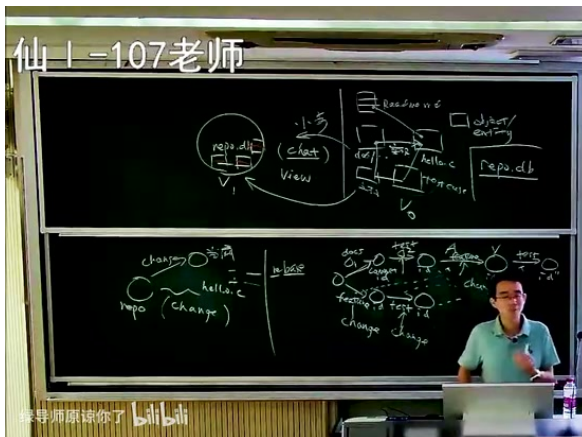
- 仿佛一直在 commit + amend。
 - ✦ 工作副本本身就是一个 commit；多数 jj 命令会自动记录文件修改，更新这个 commit。[工作副本](#)

冲突直接存进 commit (killer feature!)

- Rebase 可以直接完成，之后再统一处理冲突。
- ✦ 保存的是冲突的逻辑表示：带着冲突的 commit 仍能继续 rebase、merge。[冲突管理](#)
- ✦ 设计启发：把“尚未解决”表达成一种可保存、可继续操作的状态。

00:33:49

所以我觉得为什么你们作为AI native的人类能够淘汰掉我们这一代人，就是因为我已经形成了g的肌肉记忆了，然后比我更老的一代人他已经形成SVN的肌肉记忆了。然后当一旦人形成一个肌肉记忆以后，要把这个习惯掰过来就很困难了，对吧？比如说我现在对g是有肌肉记忆的，但是我即便是能够理解这个居势词的概念，甚至我可能理解的会比你们可能作为初学者还要更好一些。我还是可以从我的1st principle去思考它，然自己相当virtually rement这个系统。但是我即便能够理解，我也没有足够的时间再去训练另外一个肌肉记忆。所以你们是反过来的，你们跟我正好是反过来的。你们应该在你们现在还没有形成肌肉记忆的时候，就直接去用那个对的东西，就人类历史都是积累的，对吧？它它SVN实际用不下去了，才用git然后get大家发现了get的缺点以后有了，所以。



画面 00:34:00

这就是为什么 AI Native 能淘汰老登

已经形成的肌肉记忆

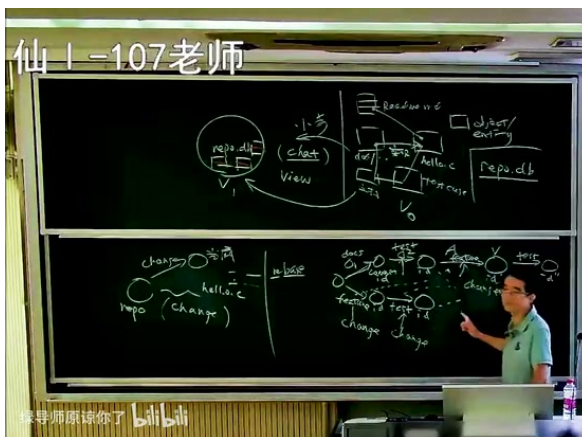
- 我已经形成 Git 的肌肉记忆了。
- 即便能理解，也没有足够的时间训练另一个肌肉记忆。
 - ✦ 熟悉的解法会锁住注意力：棋手说在寻找更优解，目光却仍围绕旧套路。[Einstellung effect, 2008 年眼动](#)
- [实验](#)

同学们的学习方向是反过来的

- **先理解正确的设计，再回望历史。**
 - 理解我们是如何一路走来 (更高效)。

00:34:50

你们可以的是先用肌肉记忆记住一个最正确的设计，然后再去理解这个设计是怎么样。就是你再回去想的时候，原来get这么蠢，对吧？在get出现的时候，你大家会想get这么先进，它是一个persistent的data structure，就是SBN。就是SBN从一开始它走从纯deep那条线就走错了，到get的时候大家觉得很先进。然后到现在的时候，你再回去看的时候，原来get也犯了一个对吧？



画面 00:35:00

这就是为什么 AI Native 能淘汰老登

已经形成的肌肉记忆

- 我已经形成 Git 的肌肉记忆了。
- 即便能理解，也没有足够的时间训练另一个肌肉记忆。
 - ✦ 熟悉的解法会锁住注意力：棋手说在寻找更优解，目光却仍围绕旧套路。[Einstellung effect, 2008 年眼动](#)
- [实验](#)

同学们的学习方向是反过来的

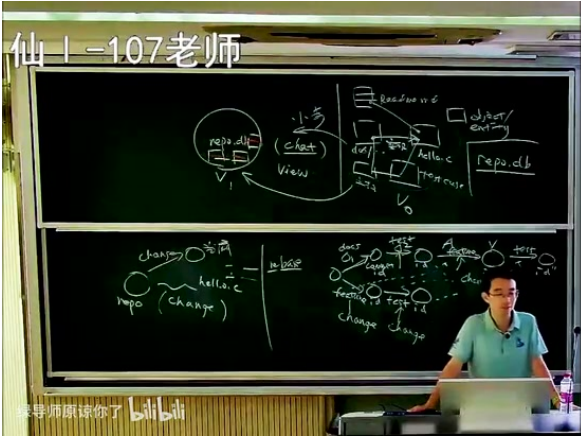
- **先理解正确的设计，再回望历史。**
 - 理解我们是如何一路走来 (更高效)。

00:35:18

犯了一些设计上，他当然这是因为最早的时候他没有想过rebase会有这么大的impact对吧？他有merge就已经很好了，就能解决他手上的问题了。所以他可能没有把这个概念模型想的更加的清楚。这是一些有意思的这个comments，也是你们学习可以告诉你们一些学习的建议。你们在学东西的时候，其实尤其在AI的辅助下，可以不断的往前走。

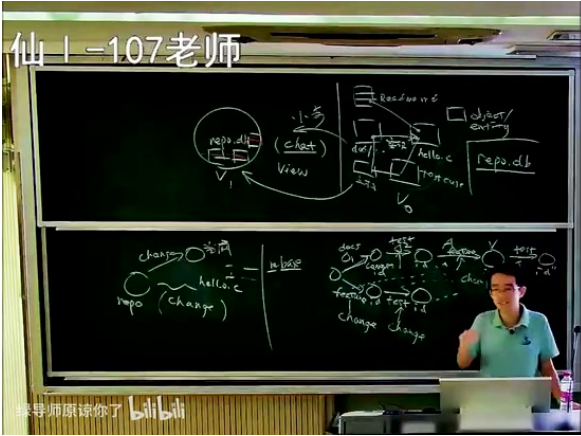
00:35:51

对，这是这个。对，好，但这门课不是软件工程，这门课是生成式软件工程。所以我们要真正要做的事儿是在agent时代再次重新设计版本管理系统。也就是说你当然可以让这个agent去用git然后让agent去用，那就就对人类来说就是一个很大的进步。但对agent来说可能进步来说相对会小一些，因为他好像很会rebase这些。那这个时候你看我们一路走来，SV有局限，git有局限。那在agent时代，我们的局限和我们要做的破掉的改进是什么？



00:36:38

刚才所有的这个version control system它都有一个假设，这个假设就是这个version control system的使用者是人类人人类有什么特点呢？人类是这个single ready的生物。我一直上操作系统课的时候讲并发程序，然变化程序。我打什么比方呢？你就把县程想象成是一个人，然后你的县程可以问共享内存。那访问共享内存的时候，两个基本的操作是load和store，或者说read和write，对吧？我可以从一个变量里读一个值，对吧？



00:37:08

我把X读到我的临时变量里，或者我可以写X等于1。然后这个操作是不能同时进行的，就是我不能说既读一个又写一个，你这样就需要一个原子指令了。然后绝大部分的这个share memory access又想象成人是什么呢？我如果想要观察这个世界，那我对不起，我手要抱在后面，我只能看。然后等我看完了，比如说我看到这个同学在这儿，我想要对他施加一个效果。比如说我想用这个粉笔导弹设击它。

00:37:36

那当我想要真正要write就写一个变量的时候，我必须要把眼睛闭起来。而我把眼睛闭起来扔的时候，注意我读到的那个值其实已经失效了，对吧？那个值已经是旧的值，就是我上一个时刻读的那在上一个时刻到这个时刻之间，可能经历了很多的事情。可能我被换cpu换下去了对吧？我被中断打断了，然后换了另外一个线程上来执行，然后执行了可能执行了一秒钟以后我才恢复过来。但我恢复过来以后，我看到的我在计算器里面看到一个X值，是我上次读X 等于1。然后这个时候我就把那个粉笔扔出去了，那很有可能就扔的就是错的，对吧？就是single spread。



00:38:14

然后当多线程的时候带来这么多线程的共享内存，就带来了无穷的问题。其实人类也是的，人类人和人之间是有无法逾越的物理屏障的对吧？你不能理解我的思想，我也不能理解我的思想。

00:38:31

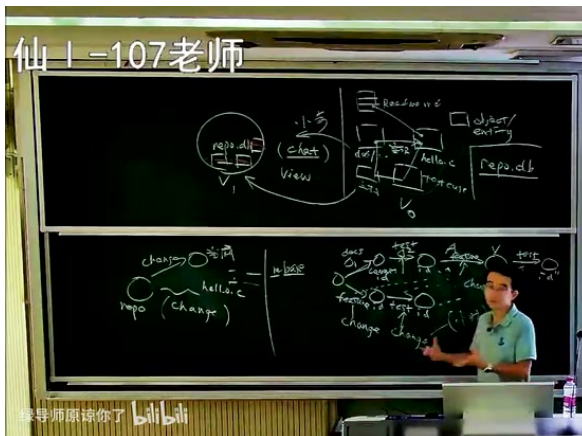
我就是个独立的个体，有分子对吧？把我很强的连在一起。OK所以我们只有一份IO设备，我们就一双一双手，一张嘴，一对眼睛。然后我们我们在比如说在编辑这个代码库的时候，以这种方式编辑代码库产生change，那就是以小时为单。

00:38:52

位的对吧？

00:38:53

小时为单位的。一个软件公司可能比如说负责一个模块的开发，可能多也就十几个人，十几个人已经是一个不小的团队了。十几个人每个人以小时为单位开发，那最多也就是以十分钟这样的对吧？十分钟这样的单位产生一个提交。



画面 00:39:00

00:39:14

那十分钟为单。

00:39:16

位产生一个提交，意味着这个速度会比人和人之间沟通的速度要慢很多，对吧？因为尤其是一个team，它工位都在一起，你们研究生的工位都在一起，你马上你再写，你要写一个东西叫一两个小时。你在写之前你肯定跟师兄说一声，师兄我要干个这个行不行？对吧？20秒钟、30秒钟，师兄说行，你去改，我们现在不再改，对吧？或者你们如果去厂里面打工了，厂里面可能每天早上都要开会，对吧？每天早上的leader都会把任务分享出来，今天我们要干什么事情。然后你你做什么你做什么你做什么你就提前在什们半小时的时间里，你就划分了一个scope，你就把这个项目开了，你领了其中一部分走，然后你就去产生change。

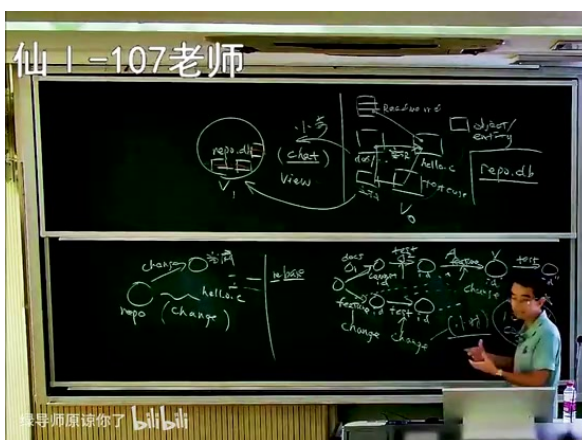
Agent 时代，再次重新设计版本管理系统

人类是“Single-Threaded”生物

- 我们只有一份 I/O 设备：一双手、一张嘴、一对眼睛

Git 基本够一个人用了

- stash: 把手头的工作放一边，先处理另一件事
- branch: 保留不同方向的尝试，切换着推进
- Jujutsu 就更好用了



画面 00:40:00

00:40:04

当然这是古法古法编程古法编程。在古法编程的时代你就看到，比如说是一个合理的对吧？是一个合理的设计。因为你你不你我刚才说的那个情况，你导师给你打电话，导师给你打电话这件事情是可能发生的。但是在你修到一半的时候，导师再给你打电话这件事情是不太可能发生的对吧？发生概率是很低的。绝大部分时候你的这个也就是st里面就一层，如果你有好多层，你完全其实挺麻烦的对，有一点麻烦你还你就需要去rebase。

00:40:39

所以你你会你尽可能的保证这个不要太深，因为你可以拒绝一个工作，你会说我手上有一个更高优先级的工作，我现在对不起，我不能干这个事儿。然后等你干完了以后，你自己可以排一个排排一个队列，对吧？所以get是给人用的。

Agent 时代，再次重新设计版本管理系统

人类是“Single-Threaded”生物

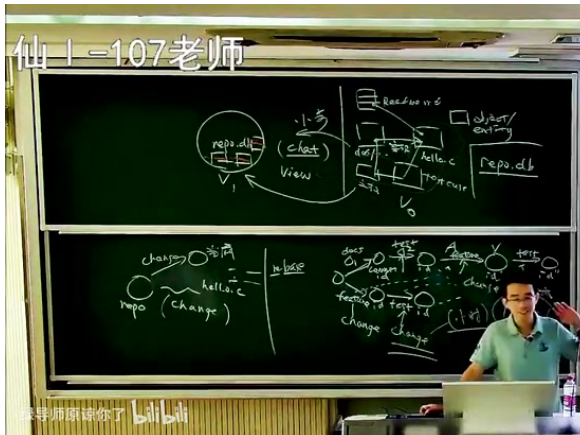
- 我们只有一份 I/O 设备：一双手、一张嘴、一对眼睛

Git 基本够一个人用了

- stash: 把手头的工作放一边，先处理另一件事
- branch: 保留不同方向的尝试，切换着推进
- Jujutsu 就更好用了

00:40:56

但是在agent的时代，这件事情变了。如果是一个agent team，单个agent他首先生产这个AS lo的速度就远远快于人，对吧？它生成A就以分钟级为单位，甚至更快。它有时候你让它fix一个东西，它在上下文是热的时候可能就极。几十秒钟对吧？这DPCV flash来干活的时候，就几秒钟就做了。



画面 00:41:00

00:41:24

那以秒级当agent工作的时候，以秒级可以产生一个原子提交，并且你可以轻易的得到一个十倍100倍的agent team的时候，get或者git是这样的，它还是跟git兼容的这样的一个为人设计的模型。就有点跟不上？确实是有一点跟不上这个速度。

00:41:45

就算是可能大型的项目，它有一些门控，就是你你你当我写完了以后，改完这个部分以后，他必须要经历一个测试的流水线，这个测试可能本身就要运行几分钟。那那可以放缓agent提交的速度。但是毫无疑问的，agent加速了这个软件开发的过程。



画面 00:42:00

00:42:04

那如果agent加速了软件开发的过程，我们应该怎么样为agent设计这个版本管理的系统，首先我们可以让agent沿用人的模型，每个agent你就想象每个agent都有一个自己的开发计算机，对吧？Agent有个自己的开发计算机，你就把single Australia的人类和agent就做一个严格一一对应的类比。你就让它有一个独立的电脑，可能是个容器，然后你就把它放到什么g点六六一点点C上，然后你就跟你就直接instruct agent说好我的这个remote仓库是在get的。你现在就是一个单人开发者，然后你现在可以从里面收到命令，然后如果你收到命令你就可以开干。所以这件事情其实是其实非常容易实现的。比如说比如说fancy。

但如果是 Agent Team (Swarm) 呢？

单个 Agent 快，Agent 还可以并行 😊

- 单个 Agent 工作的速度就显著比人更快：分钟级产生一个原子提交
- Agent 还可以并行

Git “为人设计” 的模型跟不上这个速度了

- 一个工作目录里，A 正在修改，B 就可能读到一半的结果
- 现在需要一起加速的，还有协调、验证与整合

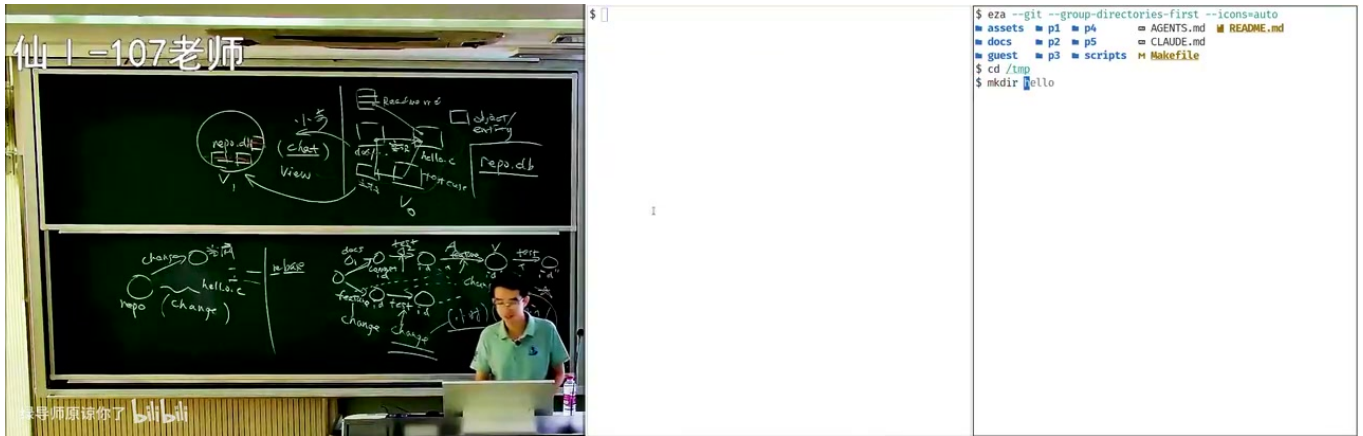
但如果是 Agent Team (Swarm) 呢？

单个 Agent 快，Agent 还可以并行 😊

- 单个 Agent 工作的速度就显著比人更快：分钟级产生一个原子提交
- Agent 还可以并行

Git “为人设计” 的模型跟不上这个速度了

- 一个工作目录里，A 正在修改，B 就可能读到一半的结果
- 现在需要一起加速的，还有协调、验证与整合



画面 00:43:00

00:43:01

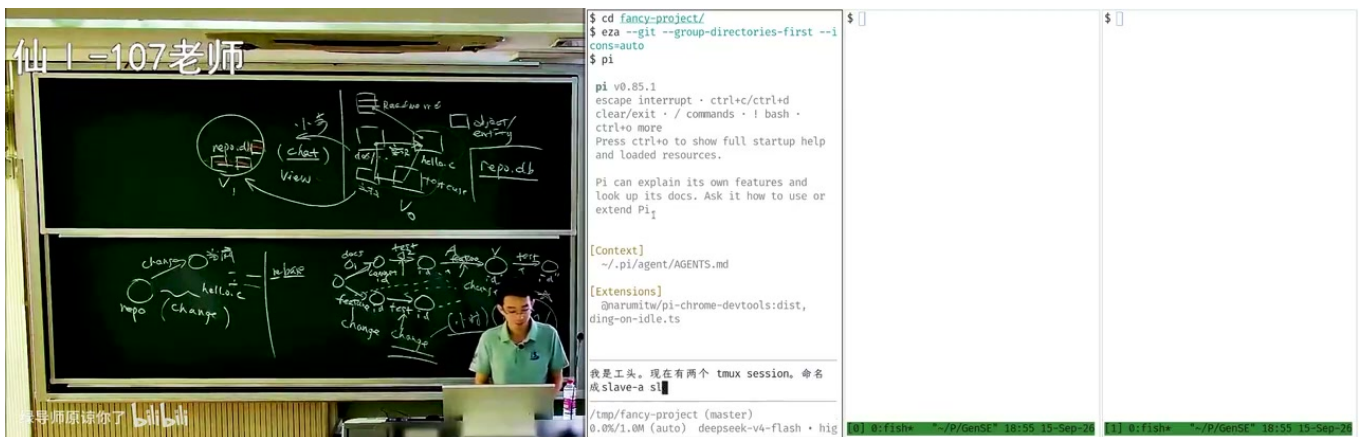
project.

00:43:03

好好我可以这样。

00:43:16

这样也可以。好，我就直接让A正在开干。然后我现在就A正时代有点什么好呢？每个人都是领导，你就是这个领导工投，然后现在有两个t Marks session，你可以想象我实际在你们实际开发的时候，这两个ti马先生都是位于两台不同的机器或者不同的容器。它物理上隔离开的，他看不到世界上面其他的人。还故意用一个可能在中国现在还能用的词，对吧？然后我现在就是master好，你看这个名字改过来了对吧？Very好好。



画面 00:44:00

00:44:14

好，我可以给他。

00:44:17

每一个都起一个派。

00:44:21

好。

00:44:26

Coding agent好，那我可以让他干什么呢？

00:44:48

Okay, anyway, it .

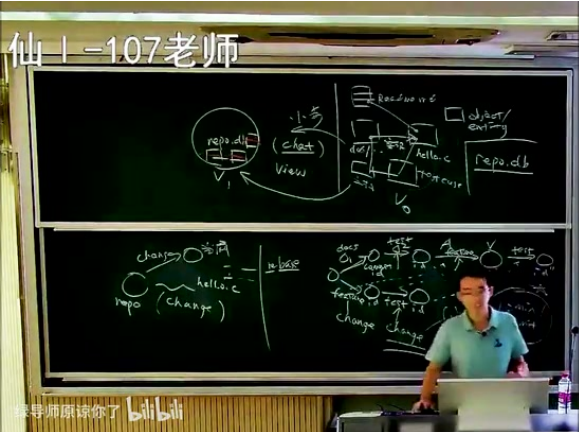
00:44:51

doesn't .

00:44:52

matter. 你看他们两个就开始工作了，开始工作了。好好他们A。

仙 I -107老师



画面 00:45:00

```
C-m
sleep 1
tmux send-keys -t slave-b 'Write some
original code, > 1000 lines. Work under
/tmp/fancy-slaves/slop-b (create it if
missing). 86 tmux send-keys -t slave-b
C-m
echo sent'

sent

Took 1.0s

$ sleep 12; for s in slave-a slave-b;
do echo "=== $s ==="; tmux capture-pane
-p -t "$s" | grep -v '^$' | tail
-12; done

Elapsed 2.0s

Steering: Do this in parallel, send key
% Alt+Up to edit all queued messages

: Working

/tmp/fancy-project (master)
*3.5k 42.1k R21k CH95.1% $0.001 0.5%/1...
```

```
cd /tmp/fancy-slaves/repo-a 86 git add
-A 66 git -c
user.name=${GIT_AUTHOR_NAME:-$(git
config user.name || echo agent)} -c
user.email=${GIT_AUTHOR_EMAIL:-$(git
config user.email || echo
agent@example.com)} commit -m "init:
repo-a" 2>&1
echo "----"; ls -la
/tmp/fancy-slaves/repo-a; git -C
/tmp/fancy-slaves/repo-a log --oneline
2>&1

... (11 earlier lines, ctrl+o to exp...
drwxrwxr-x 8 jyy jyy 260 Sep 15 18:56
.git
-rw-rw-r-- 1 jyy jyy 9 Sep 15 18:56
README.md
179f13f init: repo-a

Took 0.1s

Thinking...

: Working

~/Projects/GenSE (main)
*1382 4363 R6.0k CH93.5% $0.000 0.4%/1...
[slave-a] 8:ps>n - GenSE* 18:56 15-Sep-26
```

```
18:56 ..
drwxrwxr-x 2 jyy jyy 40 Sep 15
18:56 slop-b

Took 0.0s

Thinking...

write /tmp/fancy-slaves/slop-b/slop.py

***
fancy-slaves/slop-b

An Enterprise-Grade, Cloud-Native,
Blockchain-Ready, AI-Powered
Utility Framework for Doing Absolutely
Nothing At Scale.

Author: The Slop Consortium
License:

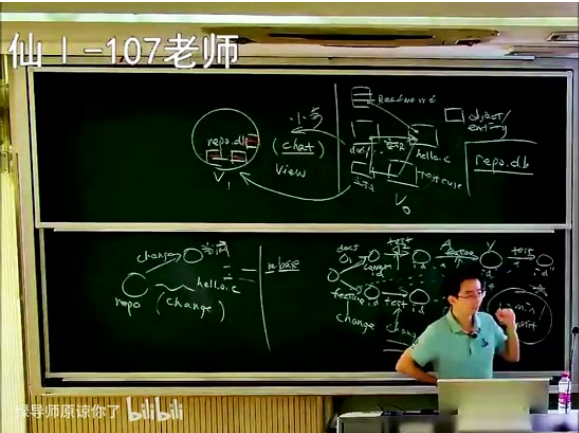
: Working

~/Projects/GenSE (main)
*174 472 R2.9k CH94.4% $0.000 0.3%/1.0...
[slave-b] 8:ps>n - GenSE* 18:56 15-Sep-26
```

00:45:14

我让他，所以你看当AI I agent变快的时候，好他他他开始写写对吧？当AI开始变快的时候，你想一想如果你的指挥发生一点点差错，或者哪怕甚至指挥就是就现在一个流行的paradigm，就是你有一个监工的一阵，然后你有好多个这个子。有些比如说比如说为了最大化模型的智力，防止AI I生成你不想要的东西，你会放两个比较强的模型，比如说由这个来设计，然后让cloud来review，然后让他们互相打架，然后达成一致，你只要开两个t box就行了。然后你的主就负责协调，就观看他们的输出和或者总结报告，然后往里面inject prompt，然后这个一个多agent就赚钱了。

仙 I -107老师



画面 00:46:00

Slave	任务	状态
slave-a	先 init fancy repo, 再写 slop (>1000 行)	: Working
slave-b	写 slop 代码 (>1000 行, 已产出 1179 行)	: Working

slave-a 的 repo 已建好 (main 分支 + 初始提交)，现在跟上 slop 任务; slave-b 文件已到 1179 行仍在续写。

Recap: 用 send-keys -t <session> '...' C-m 把指令打进对方的 pi: 靠 capture-pane | grep working 判断 slave 是否忙完——空闲才发键。避免把指令糊进正在生成的输入框，实现工头式的并行调度。

```
/tmp/fancy-project (master)
*4.1k 42.9k R39k CH96.8% $0.002 0.7%/1...
```

Slave	任务	状态
slave-a	先 init fancy repo, 再写 slop (>1000 行)	: Working
slave-b	写 slop 代码 (>1000 行, 已产出 1179 行)	: Working

slave-a 的 repo 已建好 (main 分支 + 初始提交)，现在跟上 slop 任务; slave-b 文件已到 1179 行仍在续写。

Recap: 用 send-keys -t <session> '...' C-m 把指令打进对方的 pi: 靠 capture-pane | grep working 判断 slave 是否忙完——空闲才发键。避免把指令糊进正在生成的输入框，实现工头式的并行调度。

```
/tmp/fancy-project (master)
*4.1k 42.9k R39k CH96.8% $0.002 0.7%/1...
```

```
slop_a.py 523
4 生成的 slop 模块
(240 个函数 +
20 个 Handler 类
+ 分发表)

gen_slop.p
y ~90 生成器 (固定
seed=2026, 可复
现)

- 行数远超 1000, 且 import slop_a;
slop_a.main() 可正常运行, 不是一堆坏
代码。
- 内容刻意低信号: 重复的 acc += (idx *
1) % 97, scratch.append(...), 千篇一
律的 docstring 与 try/except 包裹, 作
为 "AI 味代码" 样本。

Recap: 用固定随机种子 + 模板化生成器
批量产出 "像模像样但低信息" 的代码, 既
保证可运行 (生成后立即 import 验证)
又能精确控制规模; 随机种子让 slop 可
复现, 方便后续做 diff/评审实验。
```

```
~/Projects/GenSE (main)
*1.2k 42.5k R23k CH97.3% $0.001 0.6%/1...
[slave-a] 8:ps>n - GenSE* 18:57 15-Sep-26
```

```
达 SAD)、伪 ML、伪加密、CLI、自测试
- 已验证: self_test() 29 项全通过, CLI
各子命令可运行

'''bash
python3
/tmp/fancy-slaves/slop-b/slop.py
--version
python3
/tmp/fancy-slaves/slop-b/slop.py hello
--name slop --times 2
python3
/tmp/fancy-slaves/slop-b/slop.py slug
"Hello, GenSE 2026!"
python3
/tmp/fancy-slaves/slop-b/slop.py bench
--size 100
'''

Recap: 用 "反面教材" 的方式堆出 >2000
行可运行但刻意冗余的 slop (工厂的工厂、
永不停不到 READY 的状态机、假 ML/加
密、自测自嗨), 既能满足代码量要求,
又能当课堂素材演示 "企业级过度设计"。
```

```
~/Projects/GenSE (main)
*1.0k 418k R68k CH99.2% $0.005 2.1%/1...
[slave-b] 8:ps>n - GenSE* 18:57 15-Sep-26
```

00:46:20

然后我看。

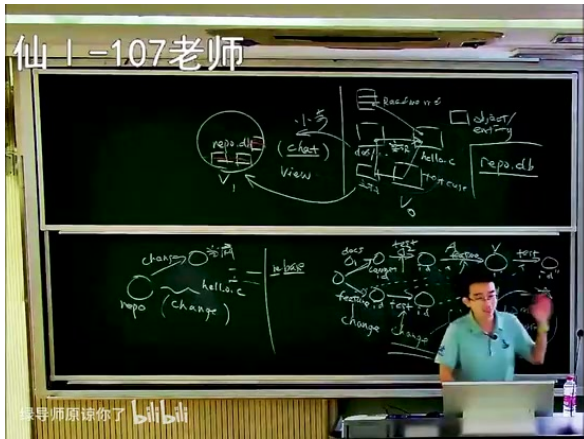
00:46:22

一下这个rebase status。然后你就发现如果大家工作在同一个g仓库上就会有一些麻烦。但如果他们工作在不同的ri上，就是完全可以的，他们可以合到对吧？他们可以和去去远端拉取别人的提交，然后冲突了，如果冲突了就会自己修复。然后在必要的时候，他甚至可以给这个master agent发消息，你们立即就可以构建出一个这样多agent的系统。

00:46:59

相应的我们今天也有一些work around，比如说get具有work tree这样的功能。所以你看我可以问他，我的老师说giwalk tree可以改进这样并行的工作方式。那么能为让这两个temark sessions都工作在work tree吗？

仙！-107老师



画面 00:47:00

把每个 Agent 当成独立开发者

每个 Agent 有自己的开发计算机，用 Remote 同步

- 这个流程能不能放到本地？
- Yes! 我们有了 `git worktree`

一个仓库，多个工作目录

- 每个 worktree 有自己的工作文件、HEAD 和 index (暂存区)
- 共享对象库和分支引用：提交在本地直接可见，无需互相 push/pull
- 给 Agent A 创建分支和工作目录：`git worktree add -b agent-a ../gensense-agent-a`
- 给 Agent B 创建分支和工作目录：`git worktree add -b agent-b ../gensense-agent-b`
- 各自在自己的目录干活：修改仍需 merge/rebase 才会汇合

3/6

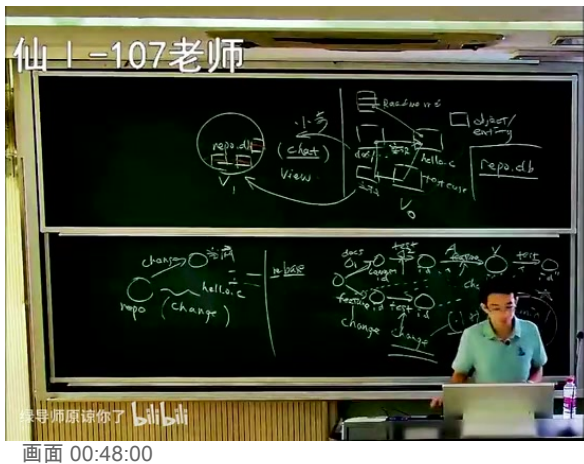
00:47:27

提问其实是关键的。

00:47:28

你们可以在各种信息，就你们甚至比如说你今天早上走到教室的时候，再看微信公众号，然后这个微信公众号上面说get，或者他提了一个什么样的sandbox container，你就可以立即让agent来帮你试一试。然后我来看看他他在做什么。这不是一个相对来说比较容易的任务吗？Ok他确实是在写写脚本，你看他把pad退出来了，应该ok了。

仙！-107老师



画面 00:48:00

分支就绪。现在让两个 slave 在新工作树里重启 pi:

```
$ for s in slave-a slave-b; do echo "$s busy=$(tmux capture-pane -p -t $s | grep -c Working)"; done
```

slave-a busy=0
slave-b busy=0

Took 0.0s

```
$ tmux send-keys -t slave-a '/quit' C-m  
tmux send-keys -t slave-b '/quit' C-m  
sleep 3  
for s in slave-a slave-b; do echo "=== $s ==="; tmux capture-pane -p -t "$s" | grep -v '^$' | tail -3; done
```

Elapsed 0.0s

Working

/tmp/fancy-project (main)
r11k 49.3k R182k CH98.9% \$0.005 1.8%/1...

	20 个 Handler 类 + 分发表
gen_slop.py	~90 生成器 (固定 seed=2026, 可复现)

- 行数远超 1000, 且 import slop_a; slop_a.main() 可正常运行, 不是一堆坏代码。

- 内容刻意压低信号: 重复的 acc += (idx * i) % 97, scratch.append(...), 千篇一律的 docstring 与 try/except 包裹, 作为 "AI 味代码" 样本。

Recap: 用固定随机种子 + 模板化生成器批量产出 "像模像样但低信息" 的代码, 既保证可运行 (生成后立即 import 验证) 又能精确控制规模: 随机种子让 slop 可复现, 方便后续做 diff/评审实验。

~/Projects/GenSE (main)
r1.2k 42.5k R23k CH97.3% \$0.001 0.6%/1...
To resume this session: pi --session 01a0 a4b5-c6d8-7368-8c3b-9286a5bc7c0a
\$ [slave-a] <fi> ~/P/GenSE 16:54 15-Sep-26

```
'''bash  
python3  
/tmp/fancy-slaves/slop-b/slop.py  
--version  
python3  
/tmp/fancy-slaves/slop-b/slop.py hello  
--name slop --times 2  
python3  
/tmp/fancy-slaves/slop-b/slop.py slug  
"Hello, GenSE 2026!"  
python3  
/tmp/fancy-slaves/slop-b/slop.py bench  
--size 100  
--'''
```

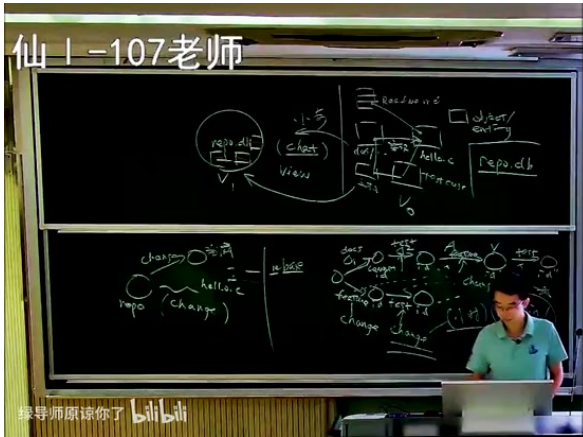
Recap: 用 "反面教材" 的方式产出 >2000 行可运行但冗余的 slop (工厂的工厂, 永远转不到 READY 的状态机, 假 ML/加密, 自测自嗨), 既能满足代码量需求, 又能当课堂素材演示 "企业级过度设计"。

~/Projects/GenSE (main)
r1.0k 418k R68k CH99.2% \$0.005 2.1%/1...
To resume this session: pi --session 01a0 a4b5-d5c2-70d4-86bd-44ad54851ff3
\$ [slave-b] <fi> ~/P/GenSE 16:59 15-Sep-26

00:48:20

Right? 这是当前的gwork tree，我们的g允许，我们创建一个工作数。然后这个工作数就是一个你可以想象成就是连接到了一个remote，就是一个get rile，然后我可以在这里面做独立的提交，在一个隔离的环境里面做独立的提交。然后大家都知道点g是什么，点击的是一个目录，对吧？点击的比如说我现在我的JSE Oh。大家都知道我的点击的是一个目录，然后它有里面有一些比如说这个logs还有objects？三种类型的对象。

仙 | -107老师



绿导师原谅你了 bilibili

画面 00:49:00

当前仓库概览

位置与结构: 当前目录
/tmp/fancy-worktrees/slave-a 不是独立仓库, 而是 /tmp/fancy-project 的一个 git worktree (.git 文件里写着 gitdir: /tmp/fancy-project/.git/worktrees/slave-a)。

Git 状态

- 分支: slave-a (HEAD 在 303bfe8)
- 主仓库: /tmp/fancy-project 在 main (69b482f)
- 另一个 worktree: slave-b (fd896c8)
- 远程: 无 (纯本地)

提交历史

```
...
303bfe8 slop: add 5000+ lines of generated filler (slave-a)
69b482f init: fancy-project
...
```

文件内容

文件	说明
README.md	"fancy-project /"

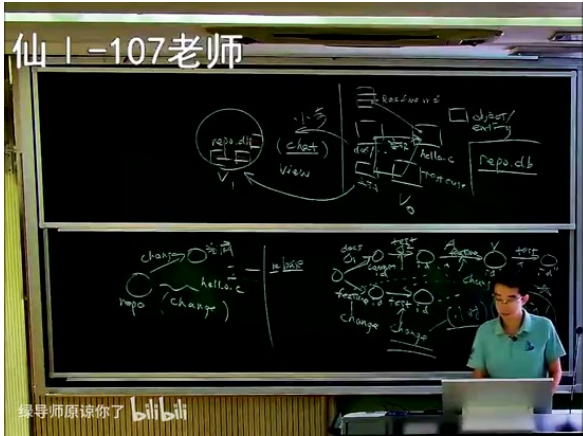
slave-a] <m"n" slave-a" 19:10 15-Sep-26

```
.rw-rw-r-- 50 jyy 15 Sep 18:59 -I .git
.rw-rw-r-- 19 jyy 15 Sep 18:59 -- .gitignore
.rw-rw-r-- 79 jyy 15 Sep 18:59 -- README.md
.rw-rw-r-- 65k jyy 15 Sep 18:57 -- slop.py
$
```

00:49:14

那work里面的点get是什么呢？点开就是一个普通的文件，是一个文本文件。然后你你如果如果你继续追问agent的话，agent会告诉你这个也是我我才知道的。就是呃除了git除了点git除了是可以是一个目录以外，它还可以是一个这样的只有这个格式的文件，然后这个文件说get dii R然后好，你看它指向了一个tree，然后这个是什么呢？这就是文件系统里面的一个文件。所以你看get最大程度上。

仙 | -107老师



绿导师原谅你了 bilibili

画面 00:50:00

当前仓库概览

位置与结构: 当前目录
/tmp/fancy-worktrees/slave-a 不是独立仓库, 而是 /tmp/fancy-project 的一个 git worktree (.git 文件里写着 gitdir: /tmp/fancy-project/.git/worktrees/slave-a)。

Git 状态

- 分支: slave-a (HEAD 在 303bfe8)
- 主仓库: /tmp/fancy-project 在 main (69b482f)
- 另一个 worktree: slave-b (fd896c8)
- 远程: 无 (纯本地)

提交历史

```
...
303bfe8 slop: add 5000+ lines of generated filler (slave-a)
69b482f init: fancy-project
...
```

文件内容

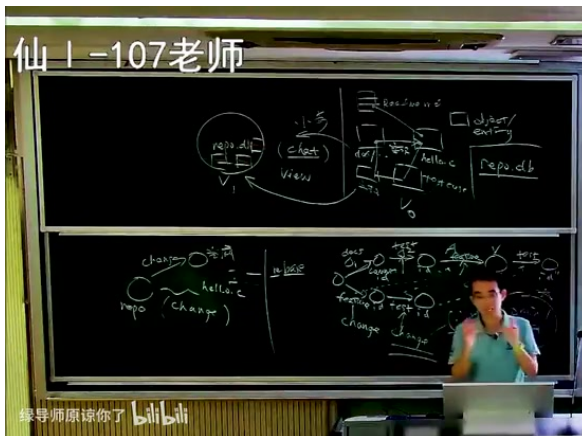
文件	说明
README.md	"fancy-project /"

slave-a] <[tmux]+ "n" slave-a" 19:11 15-Sep-26

```
$ eza --git --group-directories-first --icons=auto
# README.md # slop.py
$ eza --git --group-directories-first --icons=auto -al
.rw-rw-r-- 50 jyy 15 Sep 18:59 -I .git
.rw-rw-r-- 19 jyy 15 Sep 18:59 -- .gitignore
.rw-rw-r-- 79 jyy 15 Sep 18:59 -- README.md
.rw-rw-r-- 65k jyy 15 Sep 18:57 -- slop.py
$ eza --git --group-directories-first --icons=auto -l
.rw-rw-r-- 79 jyy 15 Sep 18:59 -- README.md
.rw-rw-r-- 65k jyy 15 Sep 18:57 -- slop.py
$ file .git
$ vi .git
$ cd /tmp/fancy-project/.git/worktrees/
$ eza --git --group-directories-first --icons=auto
# slave-a # slave-b
$
```

00:50:02

G最大程度上复用了操作系统里面的机制。这个tree就是一个这样的，它是一个小型的get目录？它是一个小型的点，给它依然有着head index locks这些一个微型的get的目录，但是它还是依附于原有的那个g目录而而生的这样一个特性，这个特性就在AI I时代突然间变得好像有点有用。我们可以在在提交直接在本地图互相互可见，不需要互相的push和push。如果你像刚才我说的，我开好几台虚拟机，然后在每台虚拟机里面放一个agent，然后我直接往那个t ma里面注入，那当然是可以的吧？这是完全可以完成的。但是workshop提供了一个更轻量的合作的方式。但是如果有比如说有冲突的修改，它仍然需要merge和rebase才能才能合才能合并，这样性能性能会更高一点。



画面 00:51:00

00:51:06

好，那我们回到这个agent team。如果我们有agent team或者一个agent swa除了，今天的work around，比如说work tree这样的机制以外，还要回头来想为什么我们需要版本控制？为什么需要版本管理？也就是说我们人类的限制，就是人类对这个呃我刚才说十分钟对吧？一个小的team哪怕是一个团队都要十分钟产生一个commit。

00:51:34

所以是因为人的限制，塑造了这个工具人之间的这个communication gap。导致了你们每天早上都要开一次会议，同步一下各个人的进展，然后再把这个任务给分下去。而且就算是人做了这样的沟通，你也都知道。你有的时候你想就你你的你的室友向你请教一个数学问题，明明你会做，你就怎么都没有办法给他讲明白，对吧？你除非你是一个很好的老师，能够共情别人，就是共情别人在哪里会能站在别人的角度去去思考，沟通的成本就是很高的。

把每个 Agent 当成独立开发者

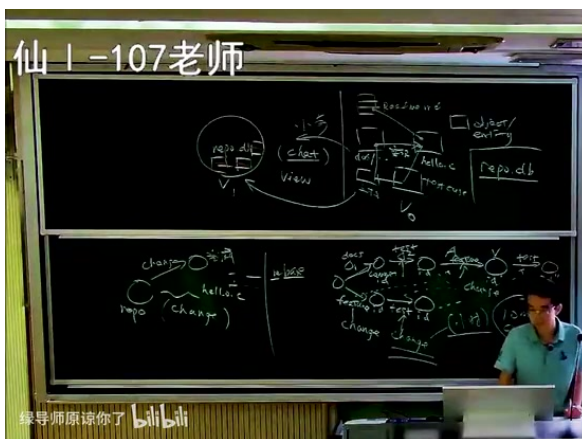
每个 Agent 有自己的开发计算机，用 Remote 同步

- 这个流程能不能放到本地？
- Yes! 我们有了 `git worktree`

一个仓库，多个工作目录

- 每个 worktree 有自己的工作文件、HEAD 和 index (暂存区)
- 共享对象库和分支引用：提交在本地直接可见，无需互相 push/pull
- 给 Agent A 创建分支和工作目录：`git worktree add -b agent-a ../gense-agent-a`
- 给 Agent B 创建分支和工作目录：`git worktree add -b agent-b ../gense-agent-b`
- 各自在自己的目录干活；修改仍需 merge/rebase 才会汇合

3/18



画面 00:52:00

00:52:13

所以也是大厂和小厂，尤其是初创公司，初创团队，为什么效率可以更高？就是当软件系统大了以后，人多了以后，他沟通的效率变低了以后，它产出的速度也就降下来了。所以可能你一个100个人的团队，他不可能达到一个十人团队效率的十倍，可能有两倍、三倍就已经挺不错了。所以团队没有办法变大，所以我们需要get的各种各样的best practice快照对吧？快照然后小步的前进，你改一个type也要做一个commit，然后你做一个什么也要一个commit然后都可以。然后这些小的commit每个都可以追溯，因为一旦出了问题以后，你需要回溯到那个人做commit个时间，然后那个人是有记忆的。

回到为什么需要版本控制

人的限制，塑造了工具

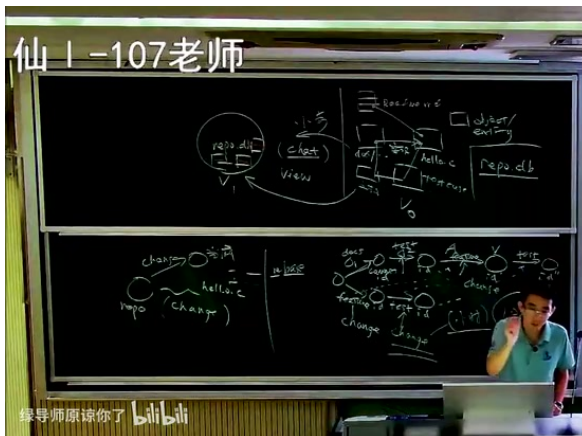
- 人的生产效率有限
- 人之间有 communication gap：实际上沟通效率极低
- 人就算沟通了，也可能各种出错
 - 快照、小步前进：bisect、blame.....

如果没有 Agent 能被开除、能坐牢？

我们也不要 blame，只要把问题默默修了就好？

- 历史的价值可以从“谁写错了”转向“哪次变化破坏了什么，怎样恢复”
- 跨领域先例：NASA 航空安全报告系统通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演

4/18



画面 00:53:00

00:53:02

开发者会回想起，那天我们当天早上开了一个会，然后我收到要做这个需求。然后，做这个需求的时候沟通发生了一个问题，他找到那个时候的沟通记录文档，然后再说我们这个地方应该怎么修正。所以你看有的 get by se 的 get blame 这些专门为人来设计的特性。但是你想一想，如果是由 agent team 或者 agent swarm 来实现你的项目，我们还需要 gblame 吗？对吧？就是因为人可以不是你这个事情我不你了，你就该被开掉了是吧？实习生你就背锅了。

00:53:41

但如果 agent 没有坐牢的被坐牢的被开除的能力的时候，这个模型就变了。我们再也不需要 blame 了，我们的项目只要前进就可以了，对吧？然后我们前进有的时候也不需要知道到底是哪一个提交引入这个错误。我只要知道这个地方错了，我也不要再也不要管它历史上是怎么造成这个错误的，我只要把这个错误修了就行了。

回到为什么需要版本控制

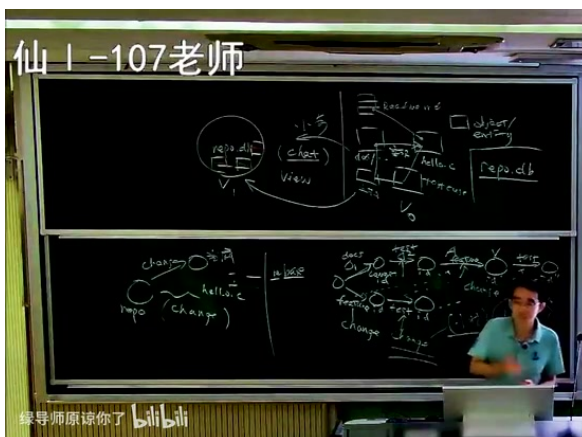
人的限制，塑造了工具

- 人的生产效率有限
- 人之间有 communication gap：实际上沟通效率极低
- 人就算沟通了，也可能各种出错
 - 快照、小步前进：bisect、blame.....

如果没有 Agent 能被开除、能坐牢？

我们也不要 blame，只要把问题默默修了就好？

- 历史的价值可以从“谁写错了”转向“哪次变化破坏了什么，怎样恢复”
- 跨领域先例：[NASA 航空安全报告系统](#)通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演



画面 00:54:00

00:54:06

因为 agent 之间的 communication，而且尤其现在更好玩的是，我们的每个 agent 是相同的模型，也就是说他们是完全相同的克隆体。同一个事情让他们来做，他们大概率会收敛的。虽然模型还是有不确定性，还是会收敛到相同的结果，对吧？同样的 prompt 的做同一件事可能有点细微的差别，但是最终的那个方向性的东西可能收敛到相同的结果，所以我们也不要 blame 了。那我们的 get 永远就是一个，或者我们的项目就永远就是一个向前进的过程。

回到为什么需要版本控制

人的限制，塑造了工具

- 人的生产效率有限
- 人之间有 communication gap：实际上沟通效率极低
- 人就算沟通了，也可能各种出错
 - 快照、小步前进：bisect、blame.....

如果没有 Agent 能被开除、能坐牢？

我们也不要 blame，只要把问题默默修了就好？

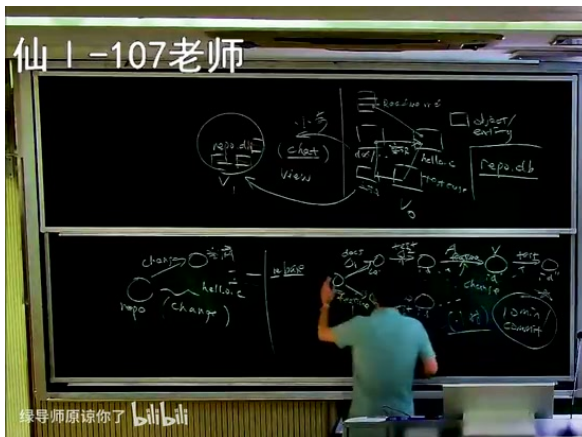
- 历史的价值可以从“谁写错了”转向“哪次变化破坏了什么，怎样恢复”
- 跨领域先例：[NASA 航空安全报告系统](#)通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演

00:54:39

每次我现在在就快照肯定还是需要的。因为比如说这个方向走错了，我们你如果没有保留历史的上的某一个 snapshot，那一下子往前走了好多以后再回去，可能它的代价比你一点一点在再往前修要好。所以保留快照肯定还是需要的那我们就是不断的大步向前。

00:55:00

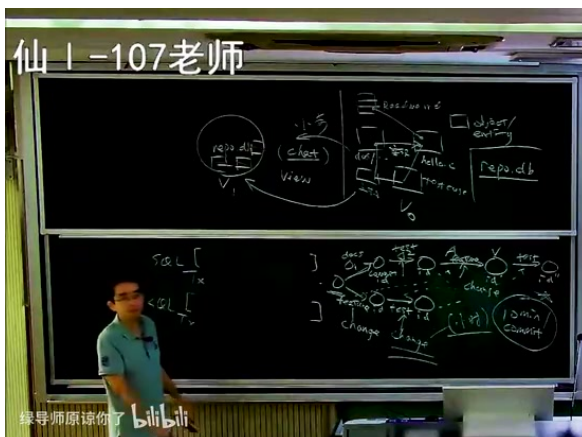
也永远几乎永远不要看过去get的这个community里面只是留了一些lessons，或者我们大部我们这个项目到底经历了什么样的演化，然后不停的就往前走，合并合并合并合并。所以说历史的价值可以从谁写错了变成我怎么把它修好。只要保存，我只要能知道我从现在开始怎么样能够往前走这件事情就可以了。所以你再再想再想这个时候你在计算机科学学到的那些知识又又起作用了。



画面 00:55:00

00:55:34

我又回到讲操作系统的时候讲并发。数据库就是就现在的这个get的模型，更像是数据库里面的并发控制。也就比如说我们有两条cy口对吧？然后这个see它都是一个transaction，它是一个很大的都是个很大的传单，可中间可以混任何代码。然后你可以先比如说你先一个select，select一个什么，你先读一下，那我这里也可以select，那我先读完以后，我可以做代码的计算。比如说我就是Python代码或jazzsquare代码。然后这个代码是图灵完备的，你永远不知道它会根据我看到结果算出什么。我甚至也可以从外面，比如说根据我当前的时间在算一个什么东西，然后算完以后我再update。



画面 00:56:00

回到为什么需要版本控制

人的限制，塑造了工具

- 人的生产效率有限
- 人之间有 communication gap：实际上沟通效率极低
- 人就算沟通了，也可能各种出错
 - 快照、小步前进：bisect、blame.....

如果没有 Agent 能被开除、能坐牢？

我们也不要 blame，只要把问题默默修了就好？

- 历史的价值可以从“谁写错了”转向“哪次变化破坏了什么，怎样恢复”
- 跨领域先例：[NASA 航空安全报告系统](#)通过自愿、保密、非惩罚性的报告收集安全隐患，帮助防止重演

主 Agent 规划，子 Agent 上锁开干

Agent 只干一件事：推进一件事，把屁股都擦干净

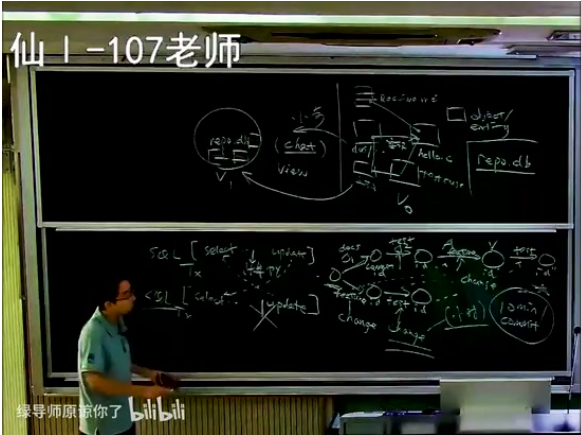
- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁，把活干了，解锁

把协调放到什么时候？

- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control：冲突检测，类似数据库
- 互斥锁：修改前先约定谁能访问这份共享状态，把冲突变成等待

00:56:27

Update, 然后update的时候就会产生冲突，也就是说这个就可能产生词组也可能产生冲突。就是说这个update可能可能如果这个update bit读它，这个bit要读它，那就会产生一个循环依赖，有可能是产生这种数据的依赖，也可能是产生就是我我我要锁你我要锁你的对象，你要锁我对象产生思索。然后这个时候就会导致这个transaction bo可能比如说这个transaction车就不能进行下去了。对不起，因为我检测到一个循环的依赖，所以对对不起，我我我这个就不能做了，我要回滚。相当于它要Roback Roback，然后其实get的模型有一点像这种乐观的并发控制。Optimistic为什么它是乐观的呢？



画面 00:57:00

主 Agent 规划，子 Agent 上锁开干

Agent 只干一件事：推进一件事，把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁，把活干了，解锁

把协调放到什么时候？

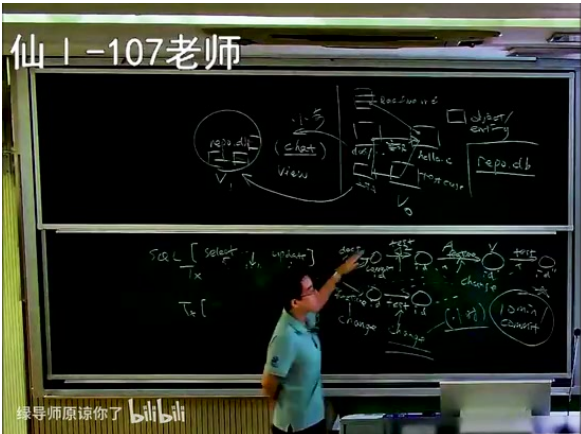
- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control：冲突检测，类似数据库
- 互斥锁：修改前先约定谁能访问这份共享状态，把冲突变成等待

00:57:14

他就说我假设这两个transaction不会冲突，对吧？我在执行的时候我会假设transaction不会冲突。这个假设在数据库系统当中是大概率成立的，为什么呢？我们想想教务系统，你们每一个同学他读的都是你读写都是你自己那块数据，对吧？你就说我选课选的是我的课，我查看课表查看的是我的课表。所以说一个全局的冲突是不太可能有的。我两个同学哪怕同时开始选课，我也是读我的写我的，这两个就可以就过去了。

00:57:52

这个的那kit也是这样，就是说我假设我这样开发，我们已经事先开过会了，今天早上开过会了。所以我可以顺着这条路先做这个，另一个人做这个，然后合并的时候大概率不会产生冲突。不管是merge也好还是rebase也好，这个都能顺利的顺利的完成。但是这。



画面 00:58:00

主 Agent 规划，子 Agent 上锁开干

Agent 只干一件事：推进一件事，把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁，把活干了，解锁

把协调放到什么时候？

- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control：冲突检测，类似数据库
- 互斥锁：修改前先约定谁能访问这份共享状态，把冲突变成等待

00:58:14

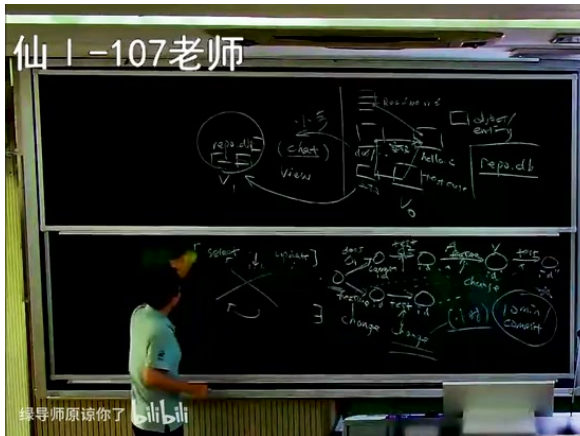
这就这是。

00:58:15

这是就古法编程时代的，就像我们see来做一个比方，但是如果我们agent的快了1000倍，对吧？Agent的快了1000倍。如果我们的这个主agent又没有协调的时候，就一下子又分了好多任务出去，他们就真的有可能更频繁的触发这个？Transaction about，然后触发这种bo在。

00:58:44

AI时代也不是什么大问题，就是agent反正出错了或者出了conflict它就修，它能修好。但是是不是在agent的时代，我们可以换一种并发控制的方式。你想想我们在操作系统里面，这是在数据库里面做的，我们在操作系统里面学的并发控制，你们是用什么做并发控制？



画面 00:59:00

主 Agent 规划，子 Agent 上锁开干

Agent 只干一件事：推进一件事，把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁，把活干了，解锁

把协调放到什么时候？

- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control：冲突检测，类似数据库
- 互斥锁：修改前先约定谁能访问这份共享状态，把冲突变成等待

00:59:04

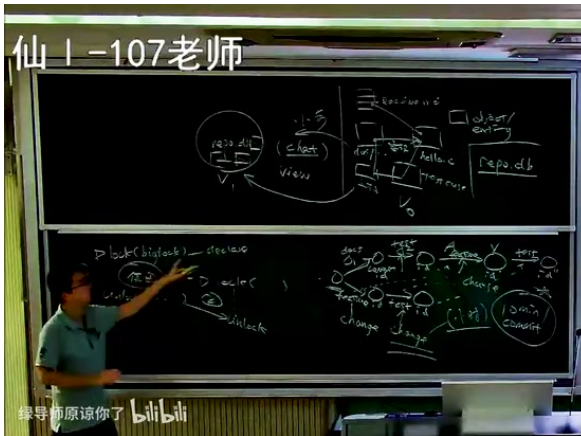
上锁锁和这个数据库里面的这个transaction是有点不一样的。你是lock一个对吧？我上课时经常说一把大手保平安，对吧？做实验你们就知道，做过实验你们就知道了当我想要访问资源可能冲突的资源的时候，我会先把它锁住，说我先想快一把锁。然后这把锁一旦快的时候，别人想要再再lock的时候，那对不起这个上锁的就必须要等到快以后，对吧？一个release快的关系建立起来就必须是先一后二，而不是这种optimistic的。大家先往前跑跑了再说。我必须是我先要知道我必须在使用这个资源之前。

00:59:55

我要declare.

00:59:57

对这个资源的使用用上锁的方式。所以看这是就是只有你你在这个操作系统课、数据课、数据库课，你真的把这些东西学会了。你回来想的时候，软件工程里面这些概念又在再再做一次。所以想想现在如果agent的动作非常快，我们其实应该用更加偏向这种悲观的并发控制。也就是说我我今天要做什么事儿，或者我有一个主agent我规划，然后规划完了以后，我就立马把这个项目的某一个部分给锁上，我可以把这个get，就就说我应该在在一个get rip或者一个快照上，我支持一个锁的操作。把这些scope说我现在锁住了，我要我要我现在做一件事需要这些，他先上锁，然后上锁以后做完以后commit，然后另外一边也可以上锁。但是如果一旦他想要干的事情被另外一个人锁上了，他就必须等到这个锁前面人释放了，他才能才能再再上锁，这样不就几乎避免了所有的冲突，对吧？



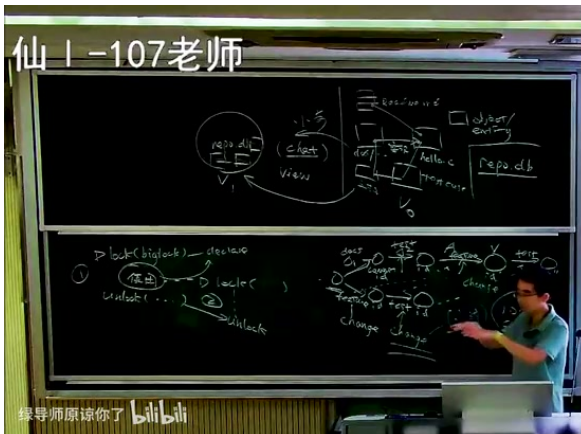
主 Agent 规划, 子 Agent 上锁开干

Agent 只干一件事: 推进一件事, 把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁, 把活干了, 解锁

把协调放到什么时候?

- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control: 冲突检测, 类似数据库
- 互斥锁: 修改前先约定谁能访问这份共享状态, 把冲突变成等待



主 Agent 规划, 子 Agent 上锁开干

Agent 只干一件事: 推进一件事, 把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁, 把活干了, 解锁

把协调放到什么时候?

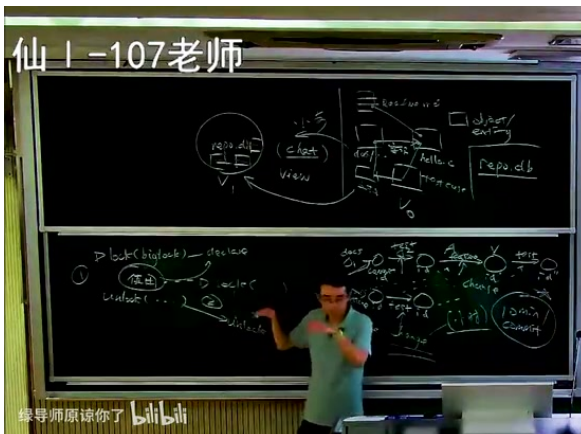
- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control: 冲突检测, 类似数据库
- 互斥锁: 修改前先约定谁能访问这份共享状态, 把冲突变成等待

01:01:05

在稳比如说打比方, 我可以在文件级上锁, 我就避免了两个A阵它会同时写同一个测试, 往同一个比如说测试或者一个配置里面做做更改。但是又因为agent数量足够多, 这个速度足够快, 我还是可以保证整体项目的进度可以往前推。因为也许还有别的agent可以继续运行, 这也就跟操作系统里面我们去实现并发程序。我经常说说如果你一把大锁保平安了对吧?

01:01:35

或者你哪怕是锁锁拆小了, 我们互斥锁, 它的本质是不要并行, 不要并发。它和这个多处理器并行是根有根本矛盾的。它不就要把一个并行能并发的程序退回一个串程序。但是当这个系统里面有好多独立的不冲突的线程在运行的时候, 我即便每一个线程都持有的一把锁, 他还是可以继去高效的。即便是悲观的, 它依然可以是高效的? 就是这些知识关系关联起来让你看清楚可能?



主 Agent 规划, 子 Agent 上锁开干

Agent 只干一件事: 推进一件事, 把屁股都擦干净

- 主 Agent 负责规划
- 子 Agent 负责上锁开干
 - Agents 完全可以直接上互斥锁, 把活干了, 解锁

把协调放到什么时候?

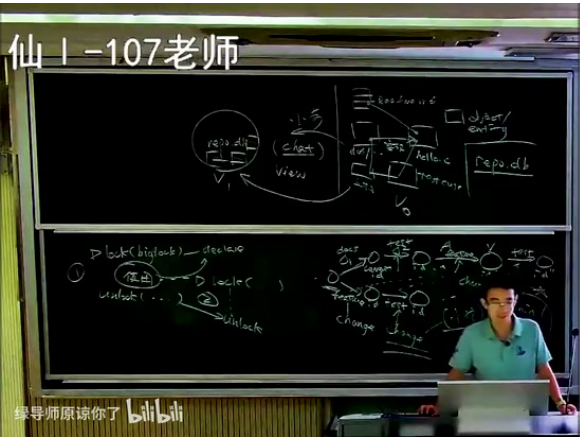
- Git/Jujutsu 像是“事后”的 Multi-Version Concurrency Control: 冲突检测, 类似数据库
- 互斥锁: 修改前先约定谁能访问这份共享状态, 把冲突变成等待

01:02:08

我觉得在agent的时代，我们又可以重新设计一个版本控制的系统。所以我觉得这就在A时代，软件工程肯定是要发生很大的变化的。你从这出发？我们要管理快照，我们要管理版本，我们要管理change，我们还要管理飞快的change？那答案是什么？可能get可能是一条线。如果我们用一种讲这样的方式来管理软件，那可能又是另外另外一条线，对吧？

01:02:36

所以你发现这个学操作系统，学数据库这些东西有用，你还是虽然痛苦对吧？你怀疑在AI I时代我学这种东西，AI I都会写了数据库对吧？这个是C口查询，我早就不写任何C口查询了。那我们还有没有可能学数据库是有用的？答案是有用的，你还是去去好好学习对吧？数据库里面有这种lock这样的机制。也是为什么我喜欢上课的原因。



画面 01:03:00

学《操作系统》、《数据库》有用了吧

“把活干完”，边界在哪里？

- 锁保护的是一件事依赖的共享状态；只锁正在写的文件，可能漏掉别人正在改的接口
- 约定：上锁 → 读取最新状态 → 修改 → 验证并留下可恢复的快照 → 解锁
- 和数据库的 advisory locks 一样，参与访问的 Agent 都要遵守协议；锁不会自动让错误的修改变正确

重新思考 Fundamental 的问题

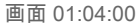
这就是为什么我喜欢上课——它会强迫我把很多 fundamental 的问题想得更清楚

01:03:05

就是我我我我如果我不来上操作系统课，我不上这个生成式软件工程课，可能就不会让我去，对吧？我在备课的时候会把这些事情在脑袋里面再过一遍，是不是地球上。所有人都都错了。因为我讲给你们的时候，我必须要共情。我的听众要考虑到你们是一个只能接受这个比较平实朴实逻辑的听众，不能直接就说这个什么事故系统是怎么设计的等等等。我必须要把那个那个最顶层的idea提炼出来，然后试图让大家共情。当然这可能我做的也不够好，你们可以在AI的帮助下再再来再来重新理解一下。

01:03:48

Ok好，版本控制，版本管理基本上其实我就我就我就讲完了对吧？从历史动机到过去讲到现在，这就次再讲到未来可能发生的事情。但是版本控制本质上还是一个工具，不管是不是，哪怕是我刚才那种，比如说基于乐一种乐观或者悲观所的版本控制，真正困难的实际上是在进度，也就是团队的管理。



真正的困难还是在**进度(团队)管理**。

AI 是一条“斩杀线”

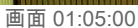
- 如果一句话就能直出高质量产品，软件工程就直接死了
- Moore's Law (也是一种 Scaling Law) 还成立，模型就有机会更强、更小、更快
- 如果一个岗位 100% 的工作都能被 AI 替代，这个人在这个岗位上的价值在哪里？
 - 从某种意义上，被斩杀的数学工作者更惨 😊

01:04:19

我们为什么现在觉得还需要软件工程？就是因为我们现在没有办法做到一句话就直出一个高质量的产品。如果斩杀线到了这个就老板有一个想法就说？就说我要做一个agent用的微信，我随口说一个或者说做一个AI时代的微信，然后这个微信是一个社交爆款，然后以后这些人社交可能也就不用微信了，就用我。

01:04:49

当然你可能有一个更具体的想法，比如说为什么你有一些思考，然后如果这时候AI直接就能出个微信级的产品，那软件工程就已经死了是吧？那我们就就呃就就就都死了。但是现在这个斩杀线暂时还没有到，所以至少没有到这个团队管理这层面，所以我们可能还还要学。但是我觉得可能也许有一天真的我也要被炸掉。因为摩尔定律本质上也是种skin，它现在还是成立的模型，肯定是更强、更小、更快。然后也许算力就有一天能够换出我们的我我们的智能，但今天至少AI还不知道怎么做团队的管理。因为每当你多一个人的时候，这个沟通的关系都是平方级的上升的。



团队协作

版本控制只是工具

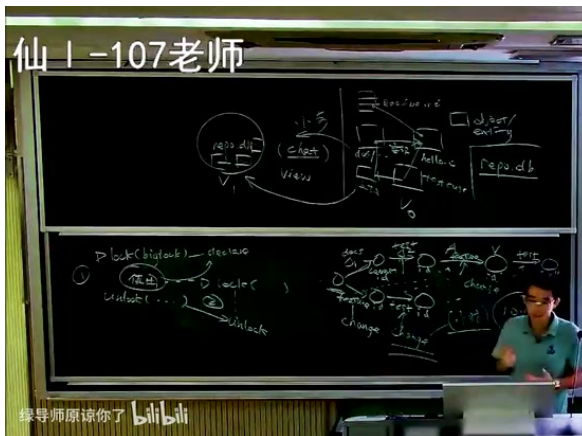
真正的困难还是在**进度(团队)管理**。

AI 是一条“斩杀线”

- 如果一句话就能直出高质量产品，软件工程就直接死了
- Moore's Law (也是一种 Scaling Law) 还成立，模型就有机会更强、更小、更快
- 如果一个岗位 100% 的工作都能被 AI 替代，这个人在这个岗位上的价值在哪里？
 - 从某种意义上，被斩杀的数学工作者更惨 😊

01:05:48

因此现在软件工程团队里面最难招的其实不是码农，是那个能把任务分解的那个team leader，对吧？也就最难招的是那种能把那种国民级的应用的那种对吧？就说我要我要去造一个微信，然后那个微信它还能够分出？我就一个100人的团队，能分出十个小团队，然后十个team leader干，最后加起来我拼起来还能成为一个团队，这个人人最稀缺。然后接下来那个能够带领小团队的人其实也很稀缺，对吧？



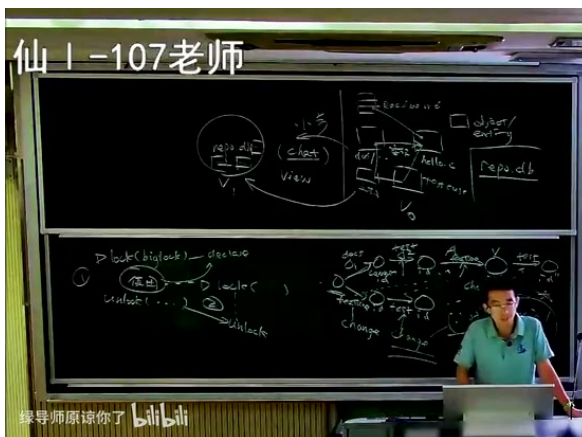
画面 01:06:00

01:06:22

这个人月神话里面，那里面就是人人之间两两之间就是平方级的沟通关系。然后就算是你把你份内的事情做好，就是因为这个平方级的沟通关系，别人也在做他的事儿。你不百分之百知道别人在做什么事儿的时候，或者你对这个整体的团队了解不是很好的时候，你你你还是就对样，最后就会各种冲突。然后你写测试我过不了，就经常会发生发生这样的事情。

01:07:00

如果我们想要管一个团队的话，或者说你们在今天在想要在毕业的时候能够具有一定的团队管理能力，你现在的就可以当成是那个主agent了，对吧？就像我刚才那样，刚刚才当然刚才我给了一个不好的示范，对吧？我说我今天创建两个slave，你你现在你你现在可以立即拥有这个deep v的这个slave。然后然后你可以去想，如果你要尽可能最大化两个或者三个或者更多agent，你应该怎么把这个任务给分解，然后你应该怎么样确定一个提交的策略，什么时候你的agent应该提交，什么时候它应该合并，对吧？然后你就可以把刚才我们讲的所有的版本控制的概念带到你的开发流程里。



画面 01:07:00

01:07:51

所以重要是这句话，版本控制是工具，它是由蒸馏了人类过去软件工程很多年的经验得到的。说我们要人类要这样开发，然后在这个工具上你可以开发出属于你自己的软件工程的直觉和经验来做。然后这里面就有好多比如说学东西还是有用的一些意外。除了比如说我可能后面再讲怎么怎么做这个软件管理。

团队的复杂性

人数增加，沟通关系平方级上升

大家都有体验：明明自己非常明白，但很难说服别人。

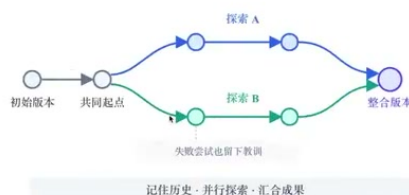
✦ n 个人，两两之间就有 $\frac{n(n-1)}{2}$ 条潜在沟通关系：5 人是 10 条，10 人是 45 条。见 Brooks 《人月神话》。

“我的部分已经好了”

- ✦ **目标不一致**：你要赶紧上线，我要趁机重构；两个人都觉得自己在帮项目
- ✦ **依赖在等待**：前端等接口，后端等需求；每个人都很忙，关键路径却没动
- ✦ **完成的定义不同**：我的代码能跑，你以为已经能交付；一集成才发现各自理解不同
- ✦ **责任落在缝隙里**：接口、测试、部署都“有人负责”，跨模块的问题却没人认领

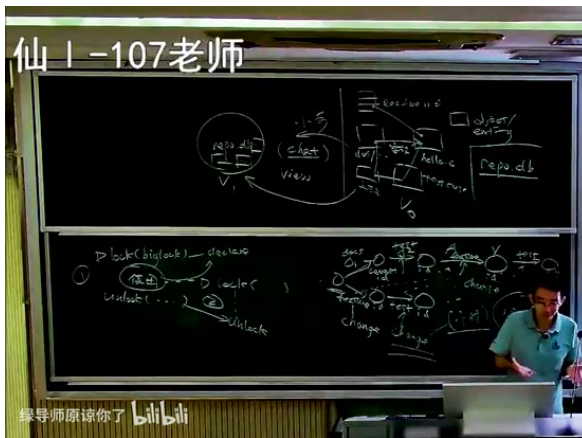
版本控制留给我们的启发

项目可以不仅是“当前状态”



技术会过时，First Principles 会留下

能够看到开发的历史、记住教训（也许会被 Agent Memory 替代），也能管理多个并行探索，并最终合并。



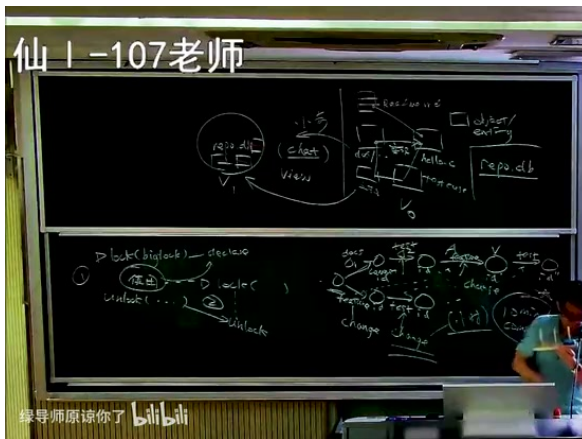
团队协作

版本控制只是工具

真正的困难还是在**进度(团队)管理**。

AI 是一条“斩杀线”

- 如果一句话就能直出高质量产品，软件工程就直接死了
- Moore's Law (也是一种 Scaling Law) 还成立，模型就有机会更强、更小、更快
- 如果一个岗位 100% 的工作都能被 AI 替代，这个人在这个岗位上的价值在哪里？
 - 从某种意义上，被斩杀的数学工作者更惨 😊



再次回到 Pico Playground

小学生的 Tower of Hanoi

- 考了个不容易的题目，一片哀嚎
- 如果为了挣¥¥¥，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

GPT: 数据库启动

“一个 API 应用、一个关系数据库、一个对象存储；需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。

01:08:24

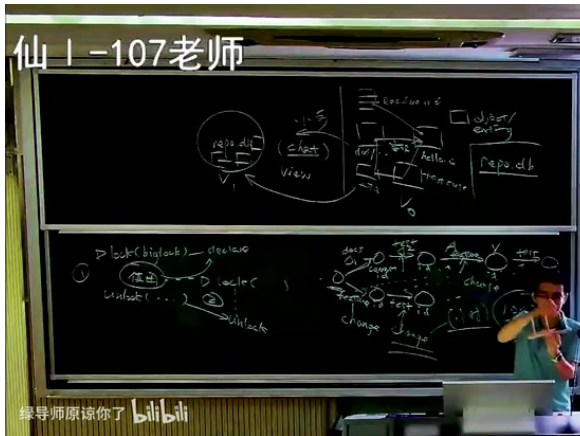
最近我看到一个蛮有意思的题目，大概说是这样的这样的我又带了这个模型对淘宝好动有很喜欢的淘宝好动作为一个面试题。这好像是应该是因为我不知道为什么就在网上刷到的是小一般是小学生考级的题目。还有塔都会低V的会还会。现在有个额外的要求，就是你的所有的盘子的移动只能往一个方向。比如说我现在两简单一点，两两个盘子，两个盘子我现在移过来，这个方向移是可以的，是可以的。这个方向仪是可以的。

01:09:10

这个方向。

01:09:11

仪也是可以的，这个方向仪也是可以，就是我要往只能往一个方向移动，当然折过来就折回去，我不允许的是就是你三个盘子的时候，你就能看出好三个盘子的时候就可以看出问题了。如果我要移三个盘子的话，我希望的是先把两个移到中间，对吧？假设两个已经移到中间了，那这时候我下一个就是从把这个盘子从这里移到这里就对了，对吧？这是汉诺塔的正常的一法，但是在这个限制下你是做不到的，在这个限制下你不能把它移到这儿，你只能往一个方向，比如说往那个方向往向移动。好，你很难想象小学生要做这样的题，对吧？然后他要输出这个，就是你想如果现在是你们觉得保研的这个机室或者是保研其实就是小学生难度。你就碰到这个题了，你要输出方案或者输出这个数字，反正也不是很大，但是你确实需要一个需要一个方案理解这个问题，对吧？

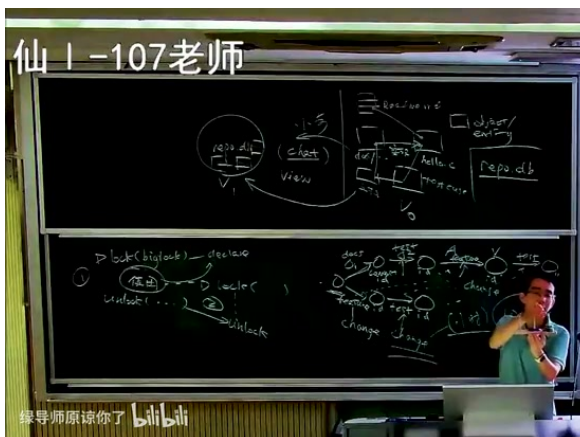


01:10:28

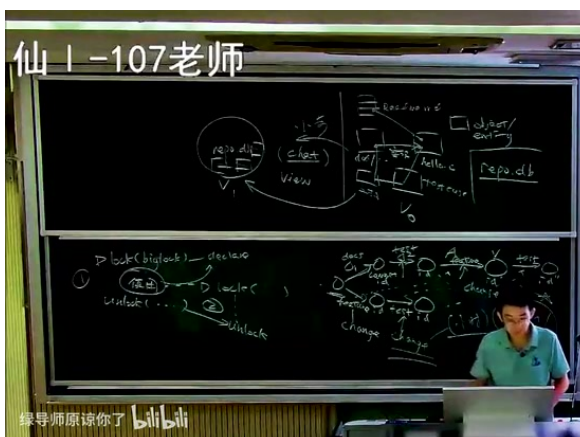
那那当然。

01:10:29

我讲的不是这个汉诺塔的故事，我讲的不是这个汉诺塔故事。这个汉诺塔的故事是你递归的时候，你现在要把它移过来，对吧？你为了移过来，你只能你还是先还是一样的。你先要把它移到这个，你需要的是先把它移到这里，然后让这个最大的盘子向右移一个，然后再把它移到这里，让向右的盘子移到这，再把它移到这里。所以就相当于是你考虑这个考虑最大的盘子，那个最优解它考虑最大盘子总是向右移一个，再向右右移一个的。好，当然我这个拆开来讲是因为我能这个吗？在这儿我准备了一个小栗子。嗯，嗯。



画面 01:11:00



再次回到 Pico Playground

小學生的 Tower of Hanoi

- 考了个不容易的题目，一片哀嚎
- 如果为了挣¥¥¥，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

- 如果为了挣 YYY，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

- 在服务器端保存小朋友的做题记录

GPT: 数据库启动

“一个 API 应用、一个关系数据库、一个对象存储；需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。

再次回到 Pico Playground

小学生的 Tower of Hanoi

- 考了个不容易的题目，一片哀嚎
- 如果为了挣¥¥¥，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

- 如果为了挣¥¥¥，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

- 在服务器端保存小朋友的做题记录

GPT: 数据库启动

“ 一个 API 应用、一个关系数据库、一个对象存储：需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。

```

$ cd pico-playground/
$ eza --git --group-directories-first --icons=auto
  apps  docs      playgrounds  scripts  AGENTS.md  = package-lock.json  📄 picopdfUI.html  📄 tokens.css
  dist  _demo_modules public      src      CLAUDE.md  = package.json       📄 README.md       tsconfig.json
$ npm run dv
npm error Missing script: "dv"
npm error
npm error To see a list of scripts, run:
npm error npm run
npm error A complete log of this run can be found in: /home/.npm/_logs/2026-09-15T11_33_16_831Z-debug-0.log
$ npm run dev

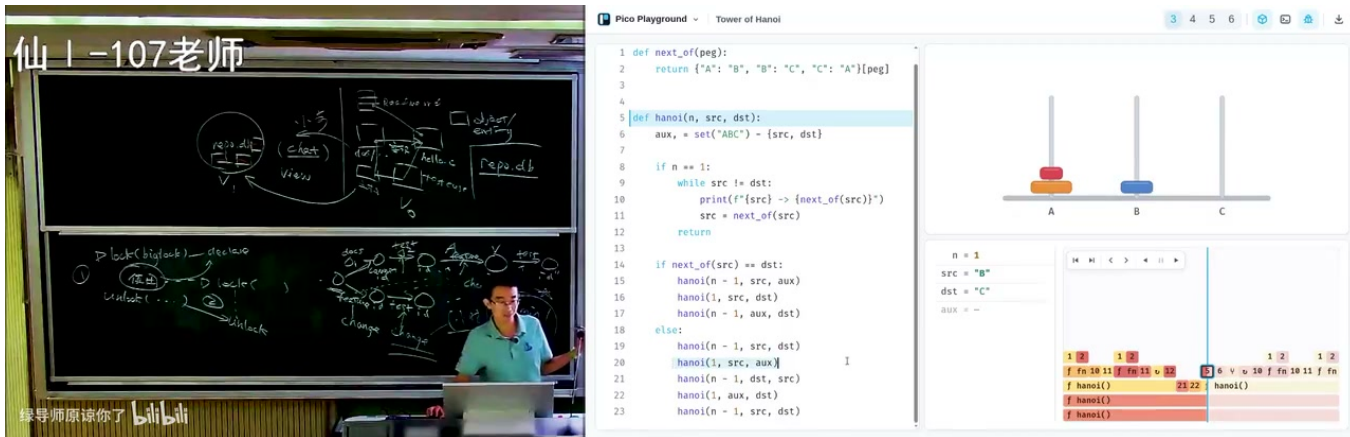
> learn-python@1.0.0 dev
> node scripts/dev.mjs

构建完成: dist/24point-ds.html (29.7 KB)
构建完成: dist/24point-gpt.html (18.6 KB)
构建完成: dist/chickenrabbit.html (24.6 KB)
构建完成: dist/cow-grass.html (33.0 KB)
构建完成: dist/hanoi.html (12.5 KB)
构建完成: dist/hello.html (4.6 KB)
构建完成: dist/picoword.html (4.0 KB)
dev server: http://localhost:5173/
24point-ds.html http://localhost:5173/dist/24point-ds.html
24point-gpt.html http://localhost:5173/dist/24point-gpt.html
chickenrabbit.html http://localhost:5173/dist/chickenrabbit.html
cow-grass.html http://localhost:5173/dist/cow-grass.html
hanoi.html http://localhost:5173/dist/hanoi.html
hello.html http://localhost:5173/dist/hello.html
picoword.html http://localhost:5173/dist/picoword.html

```

01:11:38

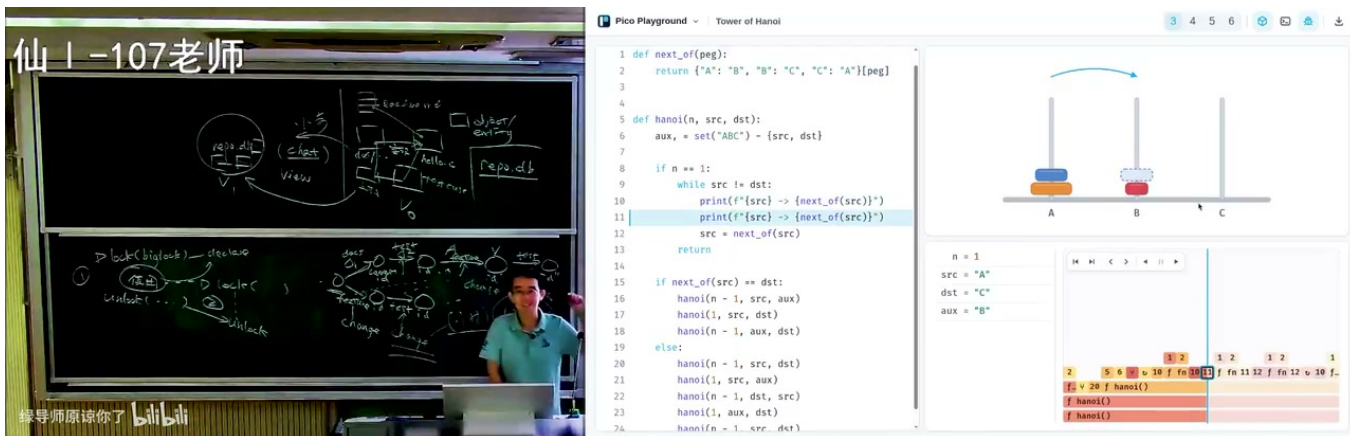
我准备这个例子，我写了一个这样的程序。好，没问题。他就是我刚才说的那个，就是我我我得把如果我的下一个目标不是目标的话，我只有最大盘子，我总是向右移一个，再向右移一个，所以它是才是最优的，就是我我最大的盘子的那个移动是最少的这就是这是不可避免的。然后剩下的就基于inductive就归纳法，我这个也是最优的，有点难对吧？你很难想象小学生竟然可以做这样的题，但这不重要，重要的是我们现在要推出这个产品，比如说我现在要把这个东西卖给小学生，那我怎我我怎么样才能最好的把这个产品卖给小学生呢？



画面 01:12:00

01:12:28

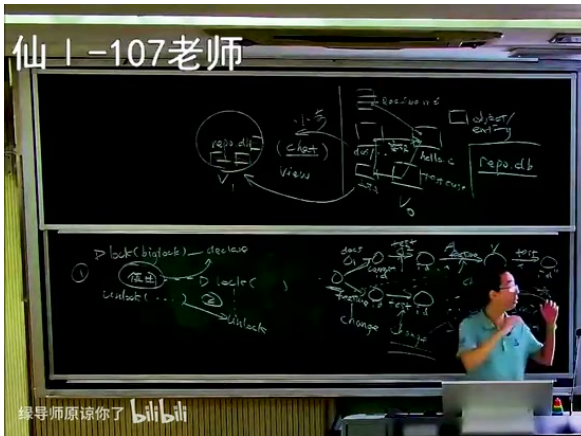
当然是提供付费的增值服务。比如说这个小学生在他可以在这里写代码，对吧？我我我这个代码是可以随时编辑的。比如说我可以再写一个，他就错了对吧？我我我的可视化是随随代码来来运动的。比如说说错了他就我就可以找到。比如说对，找到我这条语句，试图把一个蓝色的一道就小的移到移到的上，然后同学们就能找到bug。



画面 01:13:00

01:13:02

如果我要挣钱的话，我要提供增值服务。就现在还只是一个一个纯粹的你可以这个playground，然后你可以可以写代码，然后可以运行它。然后到这个时候我就要提供一个AI助教，比如说这个小朋友，当然这个是不是不太好，开玩笑的，我们还是尽可能以一个便宜的成本提供给小朋友们。当你实在做不下去的时候，你就可以点一个个叫我要助教。然后这个时候deep k的token就开始燃烧，后这样我可以收取比你比这个deep原价更贵的品费，对吧？然后然后记在你的账户上，后因为我的prompt是我独门的？我作为一个有经验的老师，我可以把我自己争馏成prompt。然后我可以敏锐的知道这个小朋友的问题在哪里，然后给他提供一个个性化的指导。



画面 01:14:00

01:14:05

好，我觉得这个时候我就涉及到要做一个信息系统了，要做一个信息系统。那做这个信息系统，因为我我有我我让学生来做这个事儿，我自己没有时间。我竟然有这么大的这个市场的钱，我竟然都不想挣这个，然后学生在设计，然后我看了那个学生和AI聊完的一版设计，然后我也让那个gp起来就是大概梳理了一下下需求。好，这个是gp的原话。

01:14:34

他说叫一个api应用，一个关系数据库，一个对象存储，然后需要异步点评时再加后台任务进程数据库关系和记录，大课程包与完整轨迹存放对象存储，任务量小时数据库任务表就够用，就是一个非常正常的。如果是你们上软件工程课，那那太正常。你看gp t它是一个语言模型，但。

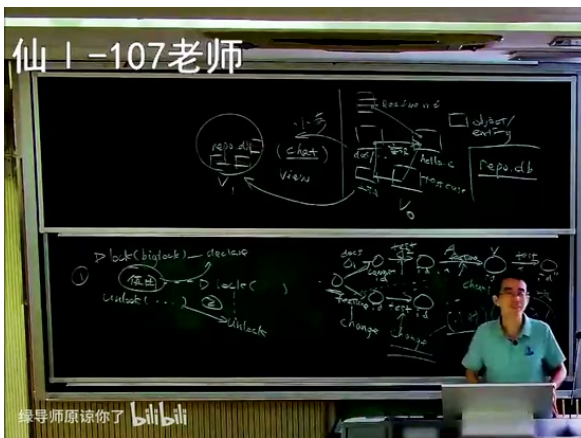
再次回到 Pico Playground

小学生的 Tower of Hanoi

- 考了个不容易的题目，一片哀嚎
- 如果为了挣 YYY，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

GPT：数据库启动

“一个 API 应用、一个关系数据库、一个对象存储；需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。”



画面 01:15:00

01:15:01

我不。

01:15:02

喜欢这个设计，为什么呢？因为我不知道你们有没有用过Overlea就跟别人协作过文档。然后overleave是有一个功能的，它可以把它当成是一个get ri导出。然后你可以直接这个时候我最喜欢这功能是因为我要给学生改论文。然后给学生改论文我就可以直接agent启动，我可以把那个那个论文克隆到本地，我就不需要忍受那个网页界面了，然后我就可以打开我的codex对吧？然后让codex去去帮我直接起给每一个点text文件都去修复里面的这个语法错误。我可以给一个我很长的prompt，让他让遵循质量就改好了。

再次回到 Pico Playground

小学生的 Tower of Hanoi

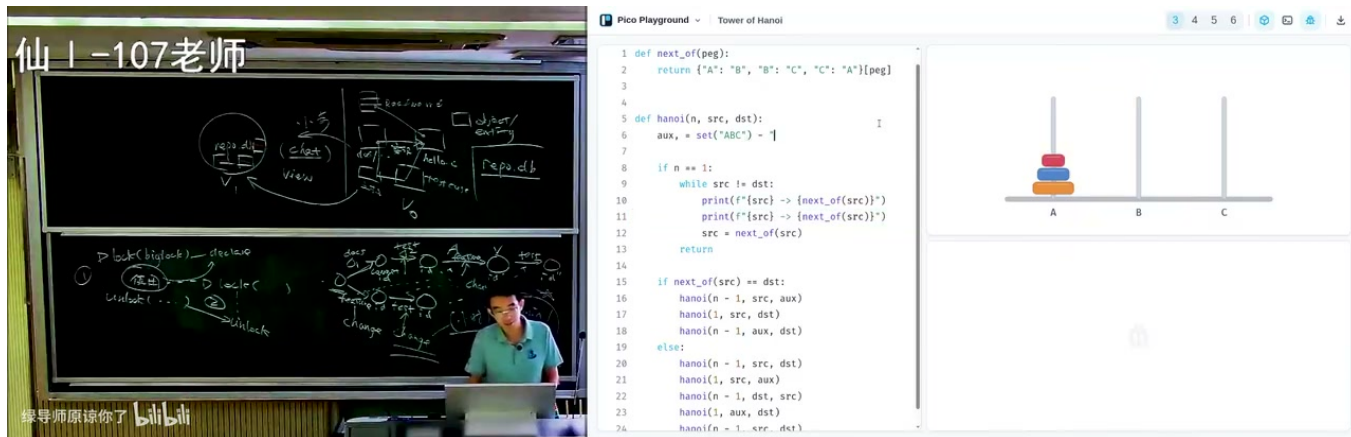
- 考了个不容易的题目，一片哀嚎
- 如果为了挣 YYY，我们当然应该提供**增值服务**
 - 在服务器端保存小朋友的做题记录

GPT：数据库启动

“一个 API 应用、一个关系数据库、一个对象存储；需要异步点评时再加后台任务进程。数据库存关系和记录，大课程包与完整轨迹放对象存储。任务量小时，数据库任务表就够用。”

01:15:46

我说学生好好结束了这个改已经改改好了。然后我们说这个picplay ground，我我我可以，我难道不是应该我在我的学生每改改一次对吧？比如说ok我不知道这样行不行，可以，这个我难道不是学生每改一次，比如说我现在idle了，或者我点运行了，或者我切换了，我就自动做一个g提交。



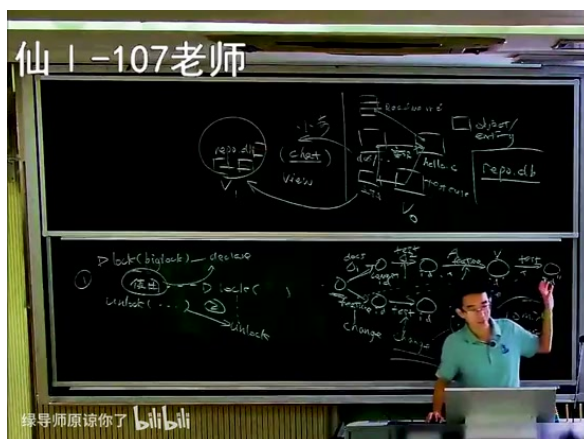
画面 01:16:00

01:16:18

然后我就可以把AI I的点评，比如说当我申请AI I助教的时候，他转圈圈，我就可以在这个get提交上面产生一个分叉，这是一个comment gt没有算的是我产生一个这个东西的成本远远大于把把剩下东西per下来的成本，就这个成本是忽略不计的。而当我用这个数据库的时候，我确实得到了一些好处。比如说总体压缩的可能更小了，效性能更高了。

01:16:53

但是这其实不是我我要的。因为我的本质，我的需求是版本管理。所以我甚至就在最早期的时候甚至最早期的时候，我可以把我的代码库和这个东西连在一起连在一起了来管，就把它放在我的代码库里。然后直到我就是我我的代码库，对，就就我的我的小朋友的代码就直接存在我的代码库里，然后我可以把探索性的在这上面创建get的历史记录。然后直到我认为整个的比如说文件目录的格式，commmessage的格式，然后我怎么样使用这些文件都都收敛了，然后我再把它迁移到数据库，这都是这都是可以的。我觉得这个还是挺好玩的。



画面 01:17:00

但 Git 不刚好适合这个吗？

GPT 也没想到

- 树状版本历史：✦做到哪一步、从哪里重试，都可以留下来
- 免费的 Diff 功能：✦前后两次提交，到底改了什么？
- Comment Commit：✦把点评关联到当时的版本：git notes 可以后补点评，不改原提交

记录“怎么做出来的”

- ✦汉诺塔只保存最终盘面，看不出孩子在哪里卡住：保存动作序列，才能回放、比较和点评。
- ✦把动作保存成逐行文本，Git 就能提供版本与差异：“哪一步值得表扬”，仍然要由应用理解。

01:17:43

你们学任何东西都是有用的，你学get对吧？是有用的。你学这个model agent的管理，你会你能够有一个机会去想到一个和别人不一样的不太一样的设计。我的好。

仙 1 - 107 老师

人就沟通了, 也可能各种出错

- 快照、小步前进 (bisect, blame...)

但如果反正没有 agent 能被开除、能坐牢, 我们**再也不要 blame**
只要把问题默默修了就好?

- Agent 只干一件事: 推进一件事, 把屁股都擦干净
- 主 Agent 负责规划
- 子 Agent 负责上设开干
- Git/Jujutsu 像是“事后”的 Multi-version concurrency control (冲突检测, 类似数据库)
- 但 Agents 完全可以直接上互斥锁, 把活干了, 解锁
- (学《操作系统》、《数据库》有用了吧)
- 这就是为什么我喜欢上课—它会强迫我把很多 fundamental 的问题讲得更清楚

团队协作

当然, 版本控制只是**工具**, 真正的困难还是在**进度 (团队) 管理**.

- 如果一句话就能直出高质量产品, 软件工程就直接**死了**
 - AI 是一条“斩杀线”
 - Moore's Law (也是一种 Scaling Law)

还成立, 模型效能更强, 更小, 更快

- 如果 100% 的工作都能被 AI 替代, 那么这个人就没有价值了
- 从某种意义上, 被斩杀的数学工作者更多 😊
- 团队的复杂性是**平方级上升**的

导师原谅你了 bilibili

p5/lect/index.md 99,8-1 61%

01:18:03

在这里。

01:18:05

还有一个好，那么接下来就是我们真的可以干一件干一件事儿了。项目的早期阶段，你肯定不能用这个 multi agent 项目的早期阶段，你你就是包工头，你先找一个应该找一个相对来说能力比较强，你能够用到的最强的 agent，然后你可以跟他多轮的对话。所以我我我要告诉大家，你们现在开始干就比什么都要好。我决定现在就开始直接 sorry，直接打开课程。

01:18:57

作业。好。

01:19:00

我不知道会发生什么，完全没有事先准备过。我还是要不开开两个好，我们有两种办法。第一种是直接然后直接回车对吧？这个其实我觉得对每个同学来说，你们都可以体验一下。当然我今天来替大家替代体验，我觉得更好的方式是Oh。

仙 - 107老师

Yanyan's wiki for TOKEN

能或记忆，用 GIC 管理实现和规则。例如，你指出“不要把报
名截止时间当成活动开始时间”。助手应当能在新会话中将这
条规则用于另一份通知。

3. 为什么要布置这个实验？

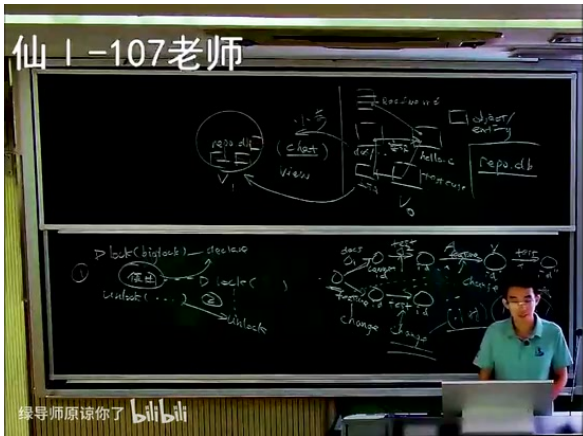
因为如果你直接把这些需求（在明确和不明确之间）给 AI
Agents，你将大概率得到一团乱麻的 AI slop——它也许的确可
以工作，但每一次修改都会消耗大量的 tokens，Agents 会在
泥潭中反复阅读代码、试错，像极了《月神话》中的软件黑
油坑，从而迫使你思考应该如何重新组织项目——但这是好
事。因为 AI 重构的能力极强，一旦你找到一个更好的架构，
瞬间就会改过。

请留存好你的项目，它可能会是你成为 AI-native 新人类
的起点——之后我们也会持续地改进它。

01:19:47

这是什么？

仙一-107老师



绿导师原谅你了 bilibili

实现一个持续为你工作的个人助手，具备以下五项基础能力。可以基于现有 Agent 配置工具、脚本和技能：语言、模型和交互形式不限。

- 个人数据库。能索引、检索你的资料，并随资料变化更新索引，回答时能定位到原文。格式不限，推荐 Markdown + 目录结构：你可以直接阅读、修改文件，也更容易观察 Agent 读了什么、写了什么，以及如何组织记忆。
- 持续关联 email 邮箱。能持续获取新邮件、关联历史往来，结合个人资料做好回复草稿。你可以编辑草稿，再直接发送；发送的应当是你确认的最终版本。重复检查邮箱不应重复处理或发送同一封邮件。
- 办理 ehall 事务。能在 ehall.nju.edu.cn 中查询信息，并完成至少一种事务的材料准备和表单填写。行为要保守：提交前展示关键字段及后果，由你明确确认；退课、撤销申请等操作尤其不能自行决定——Agent 把你的课都退了，你就完蛋了。
- 手机端联动。能方便地从手机发起任务、查看进度、编辑草稿和确认操作。可以做 App、网页，也可以借助手机已有功能。例如，在 Mac 上用 AppleScript 操纵备忘录，再通过同步与手机联动。任务不能依赖桌面端某个始终打开的对话框。
- 能力组合。上述功能应当能共享必要的上下文，完成跨工具的任务。如何组合由你决定，但至少做出一条你愿意反复使用的完整流程。例如，收到办事通知后，助手检索个人资料，整理待补材料，在手机上向你追问，准备 ehall 表单和邮件草稿；你核对后分别提交、发送，结果再归格到个人数据库。也可以设计完全不同的流程：这是一个发散的任务。组合方式本

README.md 6L, 1868B written

```
$ eza --git --group-directories-first --icons=auto
assets  p1  p4  AGENTS.md  README.md
docs    p2  p5  CLAUDE.md
guest   p3  scripts  Makefile
$ cd /tmp/lab1/
$
```

画面 01:20:00

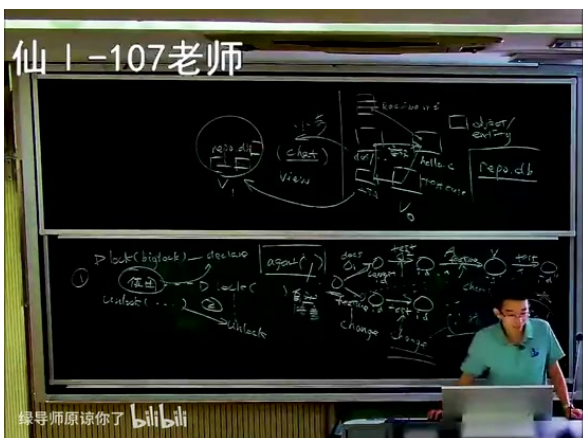
01:20:01

Hop.

01:20:11

我们来观察一下他他他他会做什么，有时候AI的直觉和人的直觉就有点不太一样。像如果你现在一线模型，它会进入一个plan mode，它会跟你再确认可能会再确认一些事实，然后他再去生成一个work flow。后如果你们看过比较，其实挺好玩的。比如像cloud code cloucode，它当它需要执行一个比较复杂的任务的时候，它会生成一段JavaScript，一段script，然后这个text数里面有一个函数叫agent。然后这个agent它里面是一个自然语言。然他用这种方式来组织播，它甚至可以有循环。就是说我有好多任务，然后他可以在循环上面来解决这个work flow o然后你看到他这就直接开干了吗？

仙一-107老师



绿导师原谅你了 bilibili

Thinking...

write /tmp/lab1/package.json

```
{
  "name": "nju-personal-assistant",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "description": "A continuously working personal assistant: personal knowledge base, smail, ehall, and mobile control.",
  "engines": {
    "node": ">=22.18"
  },
  "scripts": {
    ... (12 more lines, 22 total, ctrl+v to expand)
  }
}
```

Thinking...

write /tmp/lab1/src/core/config.ts

```
/**
 * Central configuration.
 *
 * Everything the assistant needs is described in one JSON
 file
```

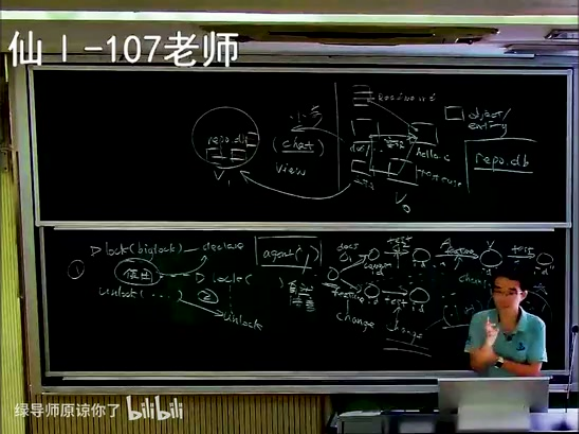
```
$ eza --git --group-directories-first --icons=auto
assets  p1  p4  AGENTS.md  README.md
docs    p2  p5  CLAUDE.md
guest   p3  scripts  Makefile
$ cd /tmp/lab1/
$
```

画面 01:21:00

01:21:18

用魔法就我对吧？只能用魔法打败魔法，这个300 tosecond实在是太恐怖了。

好，ok不管先让他搞一会儿我。然后你发现这就是我前面上面前面说这个上下文工程或者提示词工程里面最危险的一点。你提示词里面的每一句每一个词self attention都会看到它。然后看到它的时候，因为AI现在已经被训练成了prompt的舔狗，所以它有一种无可就是没有办法往外拉的倾向，需要遵循你的指令。所以当你把这个需求，尤其是这可能是一个讨论稿的需求，对吧？像我这是一个比较开放的需求。丢给他的时候，他会过度遵循这个指令，最终导致它会生成一个确实满足了指令。但是其实不是你想要的，就是你你回头看那个实验的要求，这个实验要求其实是言外之意，其实是这一句话叫实现一个持续为你工作的个人助手，这句话最重要，其他的话相对来说就没有那么重要，这些只是一些解读对吧？你作为一个好的为你工作的个人助手，你觉得12345是比较重要的那你们觉得当这每个人都有自己的观点，对你们来说，如果你想实现一个你的个人助手什么最重？



绿导师原谅你了 bilibili
画面 01:22:00

开始做一个项目

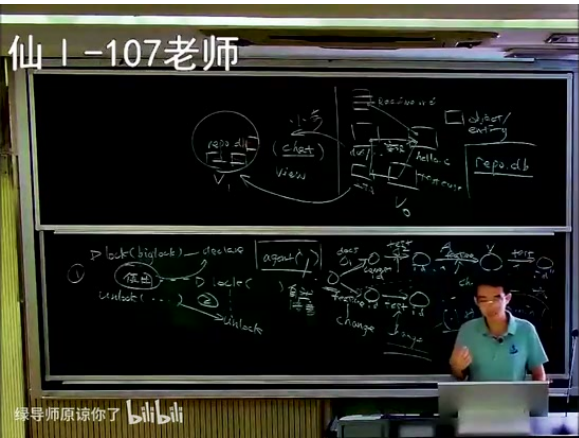
你就是包工头

不如我们直接拿 Lab 开干吧！

- 任何时候，**开始都比不开始强**

从零开始：连需求都还没对齐

- “从零开始做产品”的难度比“参与一个成熟开源项目”大 (intent-spec-impl gap)
 - 任何“确定”的东西，都可能是把对需求的**误解**固定下来
- 准则：没有什么是不能变的，能不能以不变应万变？
 - 其实就是：你的软件能否抵抗**需求的变化**？



绿导师原谅你了 bilibili
画面 01:22:26

```
$ chr
fish: Unknown command: chr
$ chromium
[13916:13916:0915/193331.796491:ERROR:ui/ozone/platform/waylan
d/gpu/wayland_surface_factory.cc:249] '--ozone-platform=waylan
d' is not compatible with Vulkan. Consider switching to '--ozo
ne-platform=x11' or disabling Vulkan
[13862:13862:0915/193332.005837:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path= /org/freedesktop/portal/desktop: org.freedesktop
.DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
$ chromium &
$ [14886:14886:0915/194047.726989:ERROR:ui/ozone/platform/wayl
and/gpu/wayland_surface_factory.cc:249] '--ozone-platform=wayl
and' is not compatible with Vulkan. Consider switching to '--o
zone-platform=x11' or disabling Vulkan
[14831:14831:0915/194047.848196:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path= /org/freedesktop/portal/desktop: org.freedesktop
.DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
$
```

Yanyan's wiki for TOKEN

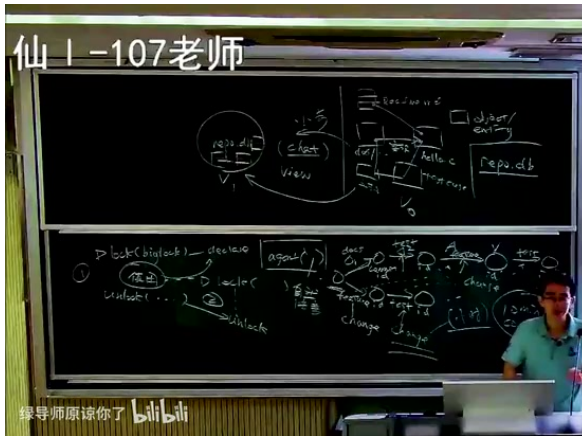
后分别提交、发送，结果再归档到个人数据库。也可以设计完
全不同的流程：这是一个发散的任务，组合方式本身就是你的
设计。

将成功流程和你的纠正保存成可阅读、可修改的脚本、技
能或记忆，用 Git 管理实现和规则。例如，你指出“不要把根
名截止时间当成活动开始时间”，助手应当能在新会话中将这
条规则用于另一份通知。

3. 为什么要布置这个实验？

因为如果你直接把这些需求（在明确和不明确之间）给 AI
Agents，你将大概率得到一团乱麻的 AI slop——它也许的确可
以工作，但每一次修改都会消耗大量的 tokens，Agents 会在
泥潭中反复阅读代码、试错，像极了《人月神话》中的软件焦
油坑，从而迫使你思考应该如何重新组织项目——但这是好
事，因为 AI 重构的能力极强，一旦你找到一个更好的架构，

就你你你实现。



画面 01:23:00

```
$ chr
fish: Unknown command: chr
$ chromium
[13916:13916:0915/193331.796491:ERROR:ui/ozone/platform/waylan
d/gpu/wayland_surface_factory.cc:249] '--ozone-platform=waylan
d' is not compatible with Vulkan. Consider switching to '--ozo
ne-platform=x11' or disabling Vulkan
[13862:13862:0915/193332.005837:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path= /org/freedesktop/portal/desktop: org.freedesktop.
DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
$ chromium &
$ [14886:14886:0915/194047.726989:ERROR:ui/ozone/platform/wayl
and/gpu/wayland_surface_factory.cc:249] '--ozone-platform=wayl
and' is not compatible with Vulkan. Consider switching to '--o
zone-platform=x11' or disabling Vulkan
[14831:14831:0915/194047.848196:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path= /org/freedesktop/portal/desktop: org.freedesktop.
DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
$
```

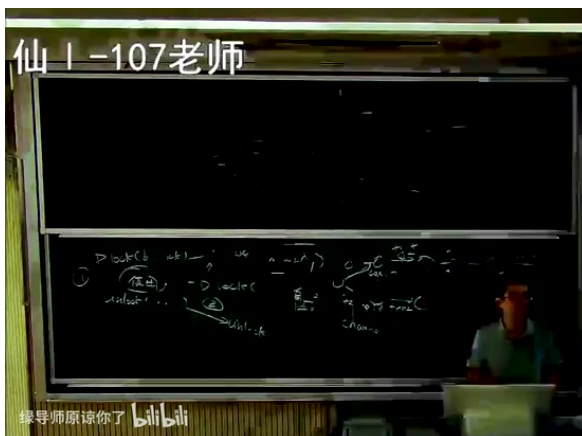


01:23:01

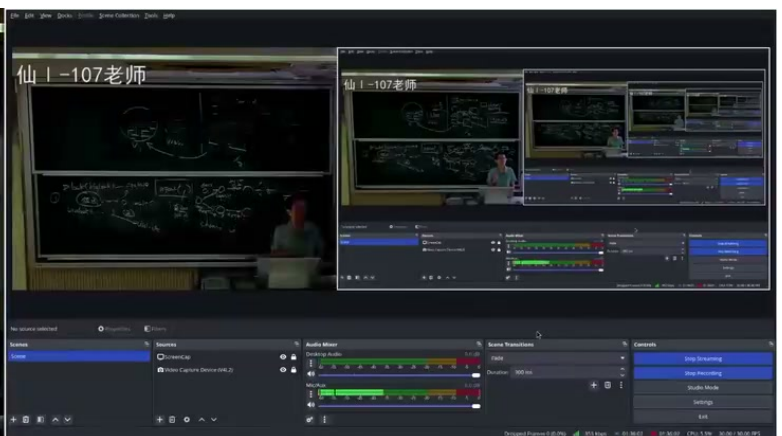
的第一个feature是什么？你想实现什么？最大的就是你在可能一个月以内遇到的最大的痛点问题，学习对吧？高效的学习和复习。因为你们要卷成绩，所以这毫无疑问是最重要的一个学习发布案。

01:23:23

你可以最好是这个agent能够自动从外网上面收集数据，然后把它以一个层次化的方式归档，然后最终并且给我一个比较好的界面，这是你最想要的。然后你看我估计他十有八九没有，对吧？我来看一下。我问他，其实我想要的是一个比较好的学习。



画面 01:23:38



01:23:55

助手。

01:23:56

能收集外部信息，除租车及外部信息。然后综合总结，根据我的进度学习，看哪部分能实现我这个功能。对吧读一读一读，然后他在做database，你看他在做database。可以这样说，就是在没有你精心设想讨论过这个时候，AI基本上就只会生成slot，就只会生成，你看它。

仙一-107老师

画面 01:24:00

100 function deepMerge<T>({base: T, patch: unknown}): T {
...
Thinking...

write /tmp/lab1/src/ehall/http-transport.ts
/**
 * Live ehall transport.
 * Real ehall services are backed by per-service HTTP endpoints and sit behind
 * the NJU unified auth (CAS). This module implements:
 * - a minimal cookie jar + redirect-following HTTP client (no deps),
 * - a CAS username/password login that yields a service session,
 * - a catalog-driven JSON query/submit adapter.
 * Because endpoint paths are deployment-specific, they live in a catalog JSON
 ... (155 more lines, 165 total, ctrl+o to expand)
} Working

/tmp/lab1
730k 451k R2.4M CH99.7% \$0.025 7.3%/1.0M (auto) deepseek-v4-f

/tmp/lab1
76.7k 41.9k R29k CH95.8% \$0.002 1.0%/1.0M (auto) deepseek-v4-
To resume this session: pi --session 01a0a4e0-c39d-7589-9385-2
cafe206cab
\$ pi

pi v0.85.1
escape interrupt · ctrl+c/ctrl+d clear/exit · / commands · !
bash · ctrl+o more
Press ctrl+o to show full startup help and loaded resources.

Pi can explain its own features and look up its docs. Ask it
how to use or extend Pi.

[Context]
~/pi/agent/AGENTS.md

[Extensions]
@narumitw/pi-chrome-devtools:dist, ding-on-idle.ts
I

现在实现到什么样子了？其实我想要的，是一个比较好的学习助手，能
收集外部xin
收集外部xin

/tmp/lab1
8.0%/1.0M (auto) deepseek-v4-flash · high

01:24:48

I smell reply没有吧？也就是说其实你要的就你脑袋里所想的那个东西。这就是我说的你的intent，你的specification，你的inten是有巨大gap的。然后但是当你一旦这个模型看到说你要实现这个的时候，他就真的是真的去实现它了。然后我觉得肯定是做了一些不好的设计。比如说虽然当我提一号的时候，我真正需要的是一个browser use agent，它能帮我各种网页上面处理L一号只是一个作为测试的出口对吧？它能把我的个人信息到一号上面。所以我可能第一步实现的不需要去在你的教务系统上面做太多的设计，你做的过度设计反而反而不好。

仙一-107老师

画面 01:25:00

86 export async function executeConfirmation(
87 db: Db,
88 deps: ExecutorDeps,
...
Thinking...

write /tmp/lab1/src/combine/workflow.ts
/**
 * The flagship cross-tool workflow.
 * "A notice arrives, the assistant does the legwork."
 * 1. Find unprocessed, actionable emails in INBOX.
 * 2. Classify each one to an ehall service.
 * 3. Extract the required materials and check which of them the personal
 * database can already evidence (with citations).
 * 4. Draft the ehall submission (profile auto-fill + a personal statement)
 ... (283 more lines, 293 total, ctrl+o to expand)
} Working

/tmp/lab1
732k 461k R3.1M CH99.8% \$0.030 8.4%/1.0M (auto) deepseek-v4-f

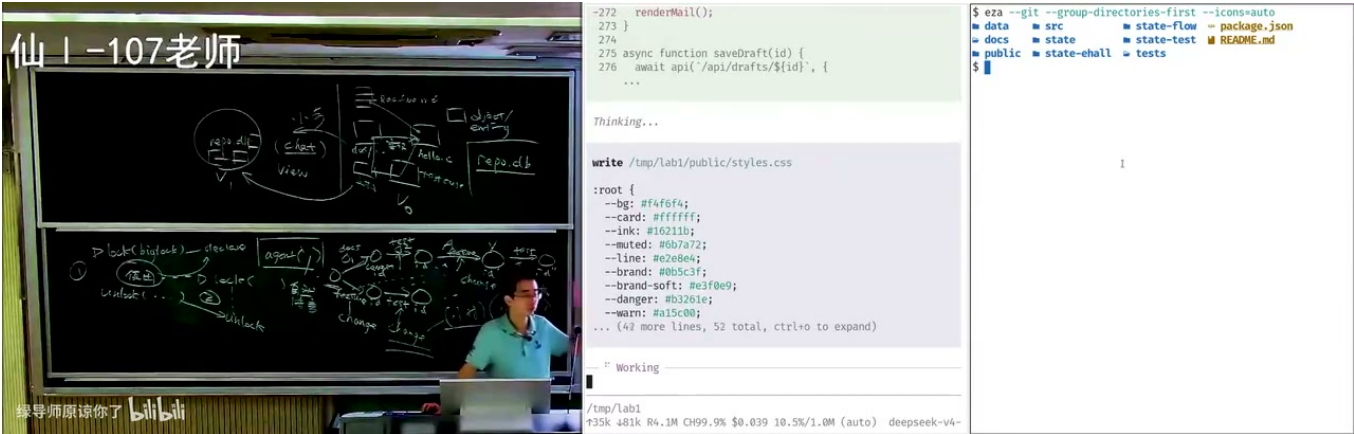
你要的三件事：收集外部信息 → 综合总结 → 根据我的进度学习。

你的需求	项目里最接近的部分	够不够
收集外部信息	email/(transport 抽象 + 等等落库 + threading) 是最完整的“外部源接入范式”：ehall/http-transport.ts 是抓网页的样板	范式有了，但只接了邮箱，没有web/RSS/arxiv源
综合总结	email/reply.ts 的composeReplyPrompt()：检索 pdb → 拼上下文 → 交给 LLM，还自带引用格式和确定性兜底 templateReply()	框架对了，但没有“大触”去调它：daemon不存在
根据我的进度学习	没有。pdb 只是你手写 md 的只读索引：tasks/confirmations 是 workflow 状态，不是学习状态	完全没有“知识点/掌握度/复习”数据模型

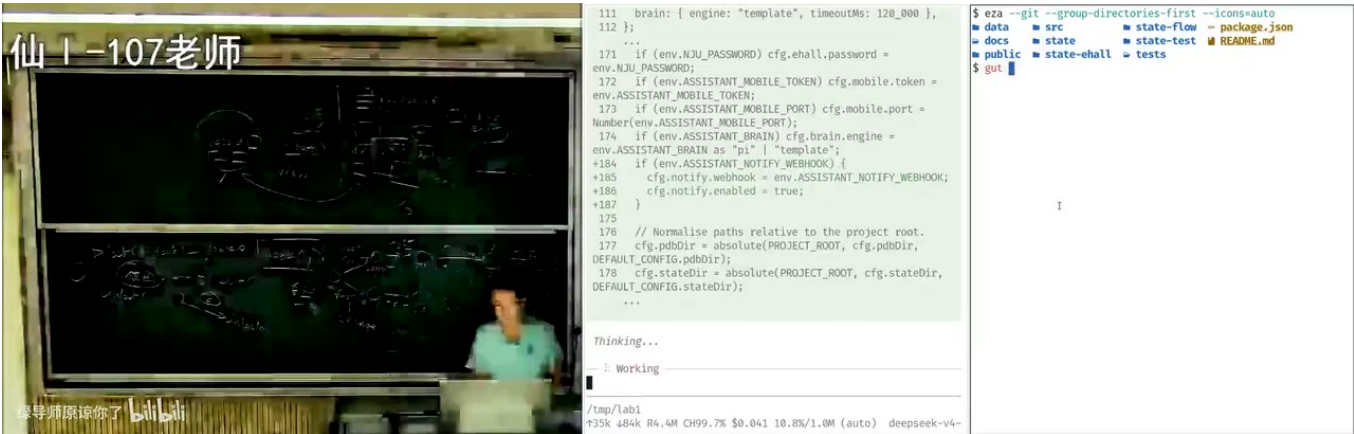
一句话：pdb (贴资料) + email (接外部源) + reply.ts 的“检索-拼上下文-生成”链路。这三块拼起来就是你想要的骨架—但缺了brain/daemon 这个执行体，也缺学习进度这一维。

01:25:42

然后这个就是和刚才我看给大家给大家看的那个例子是非常相关的。你看起来他做的所有的事情都是对的，对吧？从read me点MD来看，他做的所有的事情都是正确的，没有一个错误。但最终他得到了一个你不想要的东西。所以我看现在有没有get提交了。他甚至都没有创建一个get repository对吧？这就是你看你不跟他讲，当然我觉得可能如果是codex的话，他可能还是会初始化一个目，或者是他会问你这个项目应该怎么管理。如果比如说我是一个成熟的产品经理的话，老板跟我说这个以后我肯定会跟我的团队去沟通一下项目怎么管。



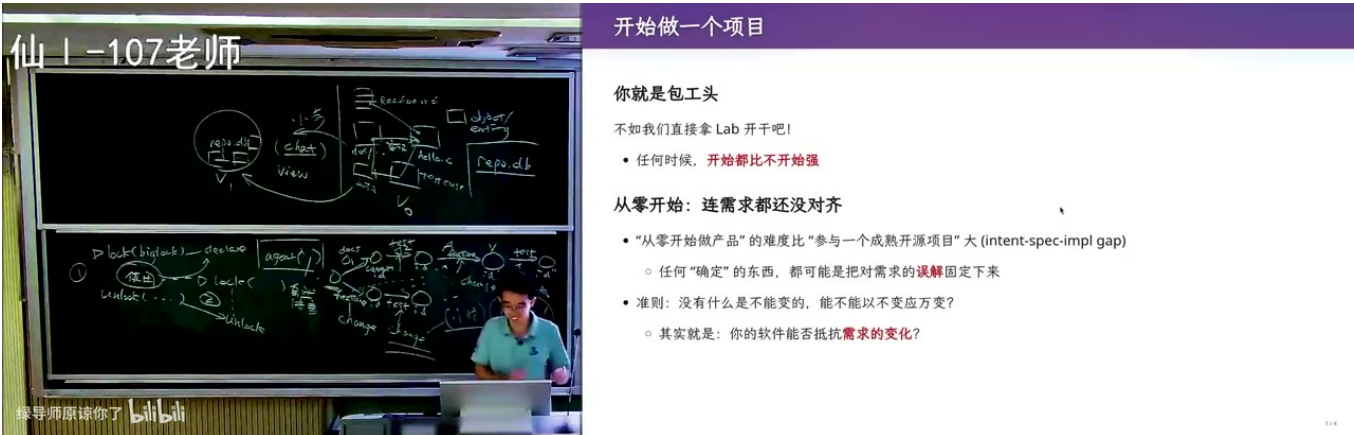
画面 01:26:00



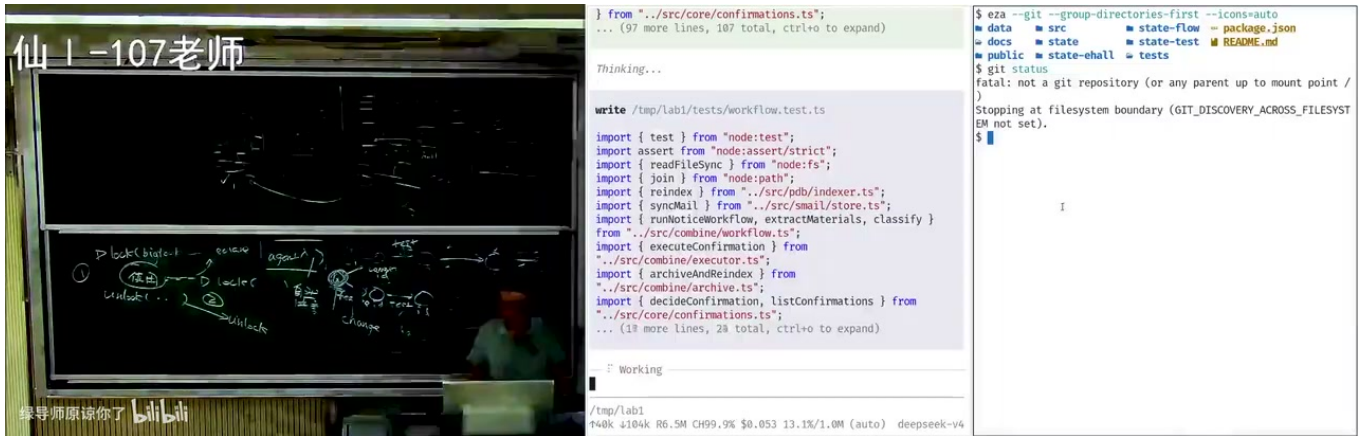
画面 01:26:08

01:26:34

就在甚至在我写下第一行代码之前，我都要讨论好协作怎么做，项目怎么管，每天什么时候开会。A这的很好玩，他就直接上手就上手就做，他觉得ok这就是一个用一次就丢掉的一个一个东西。感觉他烧了我不少，可能烧了我不少头也无所谓了。我参与deep har内测，他送了我1000块的额度，到现在还没有完全用完。所以你们在刚开始的时候还是回到项目，回到管理脑袋里面要有一个这样的模型，你不能直接把任务就丢给agent，你现在想的是先第一件事一定是创建一个空的这个。好，然后创建完了以后，你就会想你的下一步要迈向哪里，下一步我也给大家展示了，不能不能这样对吧？下一步不能是这样的，你需要的是往前再迈一小步，慢往前再迈一小步。所以你发现现在AI它从零开始做一个东西，做出来slap的难度比他到一个成熟的项目里面去做一个维护性的任务要大很多。

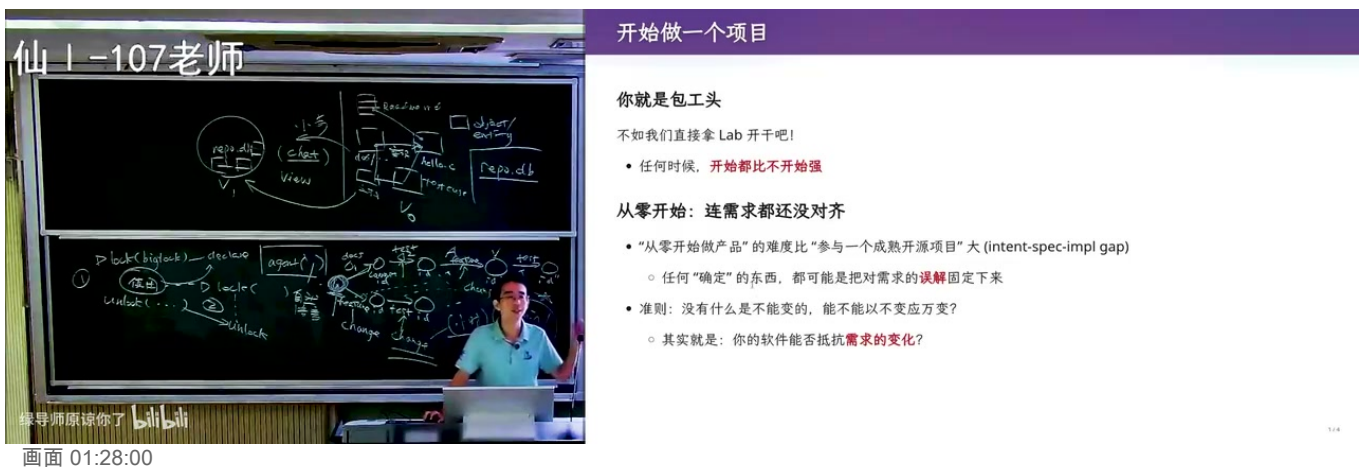


画面 01:27:00



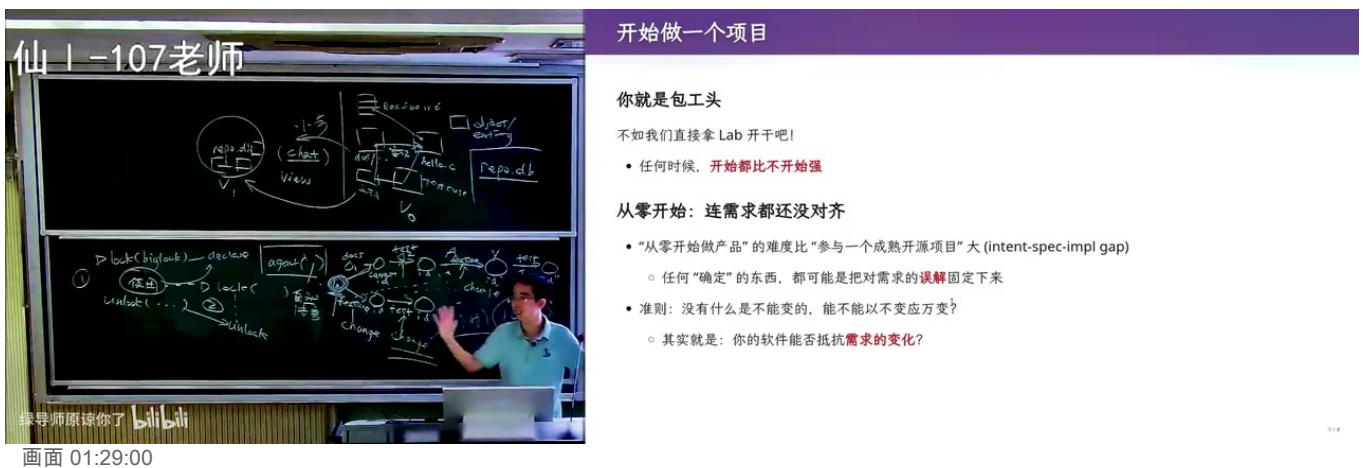
01:27:57

我们对软件工程的真正要求，或者我如果我要跟一个team或者跟一个学生去谈论我们要设计一个什么什么软件的时候，我的问题是你能不能以不变应万变。也就是说你现在软件设计一个架构或者设计一个流程方法，能不能抵抗未来需求发生根本性的变化。所以你马上就看到，在刚才那个例子里，就是数据库说说我要我要生成一个这个对吧？要生成一个挣钱的产品，去挣这个宝妈们的钱。然后我真正需要的是一个能扩展的架构。



01:28:40

当未来有别的需求来了，AI模型发生了变化。就我我其实我我在想这个软件工程的时候，或者说我的合作模式发生了变化。AI助教对吧？A助教有可能是文字的助教，有可能是图像的助教，有可能有交互，有可能没有交互。他各种各样的需求在不断变化的时候，我还有没有可能我的架构以最少的变化去应对需求的变化。



01:29:08

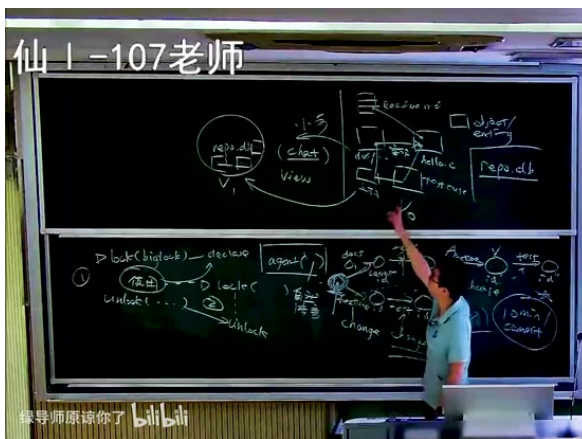
而这个恰恰是大家在软件工程课也不教这个数据库，这操作系统课也不教，对吧？大家都说这个操作系统是长这样的，你为什么今天你会看到一个这样的操作系统？为什么会看到一个这样的数据库？数据库为什么要这样做并发控制？都是有这样的一个原因。但软件工程说的最不一样的地方就是对未来需求的变化。所以如果你们要比如。

01:29:37

说这两。

01:29:38

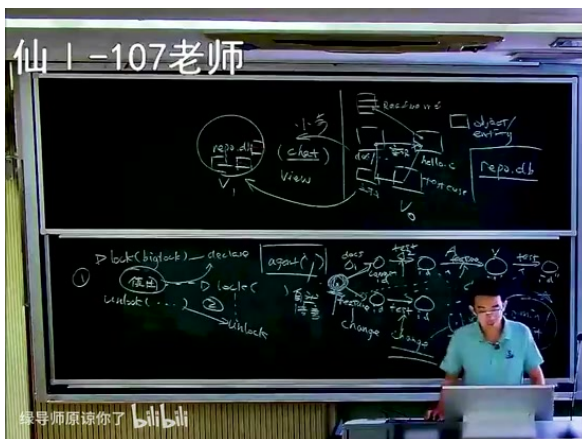
节课跟大家讲版本管理的时候，你要记得版本管理做的三件事儿，记住现在，记住历史，记住教训。这个是在agent的时代和软件工程，就古法编程的时代都成立的。你需要现在就是因为你必须要有一个reposit不管，是这个形式还是就现在的一个C或者C加加的那pyson加所有的项目你都需要把文件管起来。



画面 01:30:00

01:30:07

么你记住历史是为了有一天你这个东西不要了，对吧？或者你删掉的一个东西，你最终发现对你是有用的，你还可以把它捞回来。然后在这个提交的历史过程当中，你能看到你是怎么演进到一个点，然后发现这件事情失败了。然后这个失败了可能你会留在这儿，然后你会回到另外一个点，再开始另外一个对吧？正确的正确的路线这些东西都是在版本控制里的。然后这些lessons不停的都在警示你，你需要怎么做，对吧？你需要你怎么样从第一步开始迈出向后的一小步，那肯定显然不应该是看看他已经实现完了吗？



画面 01:30:50

Vibe Coding: 回到原点，又踩一遍坑

记住现在、记住历史、记住教训



✦ 代码记住结果，决策记录记住“为什么这样做、哪些路走不通”。

2/4

安全闸门：唯一执行入口 executeConfirmation, 状态机 pending=approved*executed 单向且带 WHERE status='approved' (天然幂等)；短语 medium=确认、high=我确认执行；所有副作用写审计日志：redactConfig() 防泄漏。

运行

```
bash
node src/cli.ts demo end-to-end # 完整离线演示 (含确认前后对比)
node --test tests/*.test.ts # 20 个用例
ASSISTANT_MOBILE_TOKEN=xxx node src/daemon/main.ts # 守护进程 + 手机端
pi # 项目已信任，自带 assistant_* 工具与技能
```

已用真实 pi 验证扩展/技能加载，并用 Chrome 实测手机端「输入短语→发送/提交→列表刷新」全流程（截图在 docs/images/）。

Recap: 核心思路是「Markdown 库 + SQLite 索引」与「唯一 confirmation 执行两门」一把搜索/同步/表单都做成了可靠的基础准备动作。任何对外副作用都必须经冻结 payload + 风险分级短语确认后由守护进程执行，从而在手机端也能安全完成跨工具流程。

/tmp/lab1
+54k +134k R15M CH99.9% \$0.086 16.9%/1.0M (auto) deepseek-v4-

```
$ eza --git --group-directories-first --icons=auto
├─ data      ── src      ── state-flow ── package.json
├─ docs      ── state  ── state-test ── README.md
├─ public    ── state-ehall ── tests
$ git status
fatal: not a git repository (or any parent up to mount point /)
Stopping at filesystem boundary (GIT_DISCOVERY_ACROSS_FILESYST
EM not set).
```

提交人民奖学。

画面 01:31:00

金申请证。

准备退课。

看有没有qi哎，他倒是做了一个提交，直接一个提交。

引擎全部是真代码，手机端是真HT服务派是集成的，模拟的部分邮箱是没有的。哎好。

呃。

画面 01:32:00

01:32:06

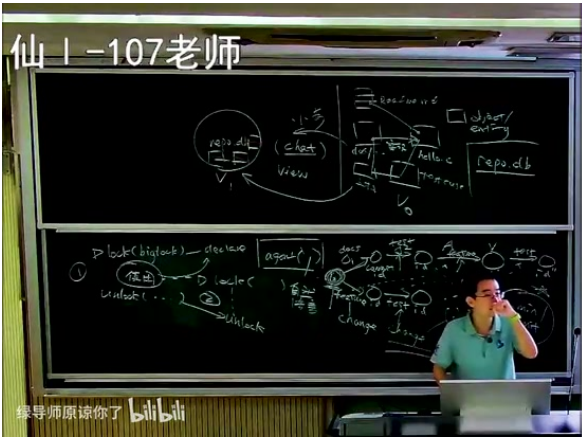
它有一个入口层，它有一个编排层，有一个能力层，还是个非常典型的架构数据层，我就不试图不试图review了，但显然它把它当成是了一个是一个业务逻辑系统来做。然后这其实是其实有非常大风险的，就是它看起来已经像一个教务系统了，这个软件已经看起来像一个教务系统。我下节课就会讲讲教务系统，而更其现在更符合主流的best practice。

01:32:41

如果你真的要从零开始构建的话，你要用一个叫MVP的paradigm叫minimum viable product或者minimal viable product有一个最小的验证版本。然后这有一句话叫。The biggest risk in product development is building something nobody wants.

01:32:59

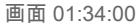
就是你做的东西，就是你看他他做的东西就是我不要的对吧？这个东西绝大部分东西其实不是我要的。我刚才在他登代码的时候我解释了，我其实想要的是是什么，对吧？所以如果你要做一个最小的版本，其实他逼迫的是你要想清楚什么样的需求才是你绝对要的对吧？刚才我已经说了，如果我要一个课程的，如果我要做一个人的助手，我最想要的是他能有让我提升我学习效率的能力和提升我工作效率的能力。



画面 01:33:00

01:33:39

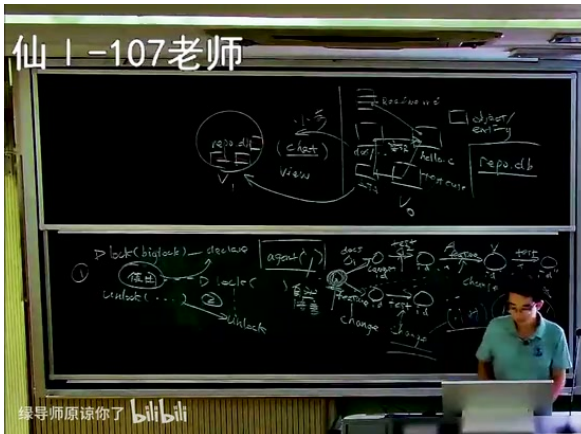
你总结出了这两条以后，你可能在第一个这个MVP里面没有任何的数据库，对吧？因为你会想到在任何时候，当你文件系统膨胀的时候，你都可以说跟着直接说好，按照这个数据的组织帮我提炼出一个数据库，这个问题就解决了。所以你不需要在刚开始的时候就做数据库。因为一旦你带上一个数据库，你每一次数据库重构都需要让agent做这个database的迁移。但文件系统不一样，文件系统你自己就可以在你的这个文件管理器里去调整它的目录结构，逐渐梳理到你舒服的一个结构上。你看看你在里面调文件和你去调这个database的schema，你的心智负担是不一样的。人类已经训练过，对吧？人类对直观的东西已经训练的非常舒服了。



- ✦ 先选一个最值得验证的假设，再做能验证它的最小版本。(Eric Ries: MVP)

画面 01:35:00

- ✦ 先选一个最值得验证的假设，再做能验证它的最小版本。(Eric Ries: MVP)

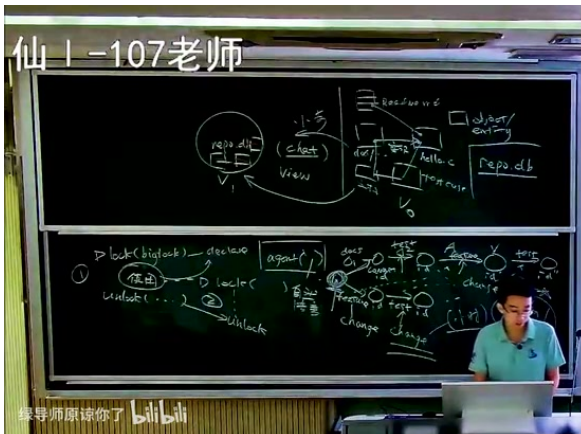


绿导师原谅你了 bilibili

画面 01:35:55



关键机制



绿导师原谅你了 bilibili

画面 01:36:00

unicode61 切成 token.

- 单一执行网关: 任何对外动作都是 executeConfirmation, 状态机 pending → approved → executed 单向, markExecuted 带 WHERE status='approved', 并发/重试等。
- 发送前冻结: 确认时把主题/收件人/正文 (或表单字段) 写进 confirmation payload, 执行时读 payload 而非草稿——发出去的就是你确认的那一版。
- 幂等去重: 邮件 Message-ID UNIQUE + uidvalidity/last_uid 增量; 草稿 idempotency_key; 流程 workflow_runs(message_id) 只处理一次。
- 风险分级: 服务元数据 low/medium/high, 标题命中 退课|撤销|注销|drop_ 自动升级为 high; medium 确认, high 输 我确认执行。
- 无常驻对话: 手机提交的任务落 tasks 表, daemon 原子领取执行, 关掉手机跑; PWA 用 Bearer token + SSE 推送。

组合流程 (combine/workflow.ts)

未处理通知 → 打分 → 映射 ehall 服务 → 抽取材料清单 → pdb 核对 (有依据/待确认)

生成陈述 + 填 ehall 表单 → confirmation A

生成回复草稿并冻结 → confirmation B

执行 → 收稿/邮件号

data/pdb/notices/YYYY-MM.md → 重建索引 → 下次可被索引引用

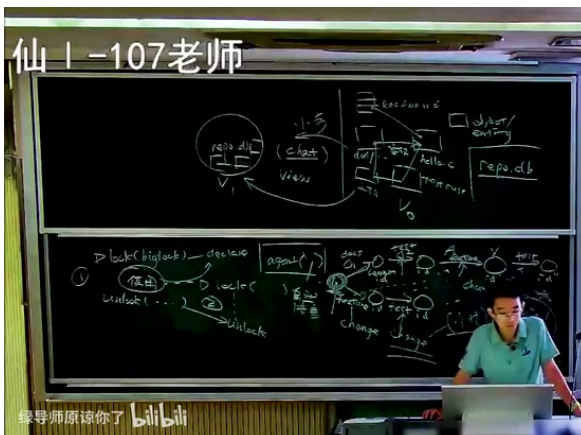
Recap: 采用分层阅读法 (入口→编辑→能力→核心→数据) 梳理代码, 用 docs/DESIGN.md 的架构图作骨架并对照源码的 schema, executor, worker 校验, 最终画出覆盖模块依赖、单一确认网关和组合流程的 ASCII 图。核心洞见是“Markdown 真源 + SQLite 状态 + 唯一执行网关”这三条主线撑起全部五项能力。

那么, 因为我很喜欢我d

```
/tmp/lab1 (master)
t6.7k 42.1k R26k CH91.6% $0.002 1.2%/1.0M (auto)
deepseek-v4-flash • high
```

01:36:49

然后我们来看一下它的设计, 它有一个system点DB。他要把个人资料放到这里, 邮件归档也放到这里, 索引草稿仍然在在低避。现在看起来还不会引起很大的问题, 因为绝大部分都是这样的数据库, 确实应该不会引起特别大的大的问题。但是随着这个项目的不断的进展, 你就会你会发现你有一种开发方式是这样, 就是你上来就是一个大的, 然后好几大步就把需求都实现完, 然后就拖着这个技术债不断的往后迭代, 往后加功能。还有一个是你从最小的就得到一个特别精巧的价格。我举个我为什么要举刚才那个例子? 就当你发现我用get管理这个学生的提交以后, 很多事情都变得非常容易。



绿导师原谅你了 bilibili

画面 01:37:00

于是原则变成: 云端同步「记录」, SQLite 是每台设备从记录派生出来的本地缓存。

数据分类: 哪些能上云

数据	位置	能同步吗	理由
个人资料	data/pdb/**/*.*.md	✓ 双向	纯文本, 天然云友好; 含 notices/ 归档
邮件归档	需新增 mail/**	✓ 单向 (IMAP→云)	原件 .eml+附件, 按 Message-ID 命名, 可重放
索引	pdb_docs/chunks/fts	✗	可由 Markdown 重建, 同步它没意义还易冲突
草稿/确认/任务/审计	state/assistant.db	⚠ 只读像只读	权威状态单写者; 可导出 Markdown/JSONL 供他设备看
IMAP 游标	同上	✗	设备本地, 共享会导致漏拉/重复拉
凭据	secrets.json	✗	已被 redactConfig 保护, 绝不上云

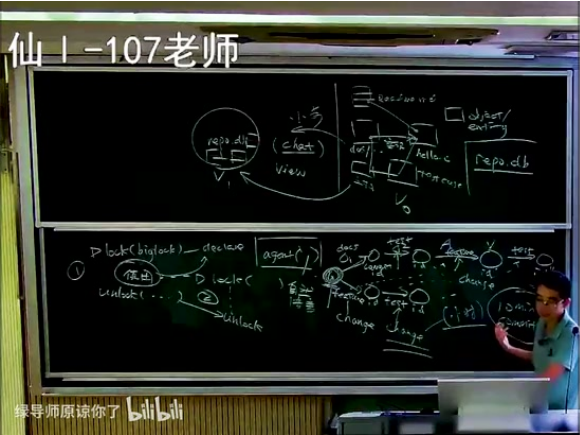
插入云后的架构

云端 (WebDAV/OneDrive/坚果云)

pull/push etag-hash-mtime 只读镜像

新层 src/cloud/ transport.ts 适配器: webdav | graph | s3 | local-mount

而且我自己控制起来也会轻松轻松很多。直到我遇到性能瓶颈之前，我都不想要把它迁移到任何一种数据库。然后这也意味着其实你早期的prompt对整个项目的走向的影响是最大的。你每一次就你最早的那个我直接把reme给他，还说我我要做一个MVP，这都会对项目造成很大的影响。我下一节课就周日就会进入软件架构和构造的内容。好，今天下课。



需要改的具体位置

1. 配置: src/core/config.ts 加 cloud: { enabled, transport, root, remoteDir, pollMs }; pdbDir 允许指向云盘挂载点。
2. 新增 src/core/db.ts 表: cloud_state(path, hash, etag, mtime, dir, updated_at) 存同步游标, 本地库, 别放云上。
3. src/combine/archive.ts 必须改: 现在 archiveEntry 是 append 到 notices/YYYY-MM.md。两台设备同时归档必冲突。改成 一事一文件 notices/2025-09/<confirmationId>.md。用确认 ID 命名 + 确定性 + 幂等 + 天然无冲突。还能被 pdb 当独立文档建索引。
4. 新增 src/cloud/exporter.ts: 把 small 收到的信写成 mail/<folder>/<messageId>.eml + blobs/, 命名用 Message-ID (已有 UNIQUE 约束思路), 重跑幂等。
5. 新增 src/daemon/tick: cloud.sync() 后若 pdb 有新文件 → reindex; 再用 SSE 通知手机端。
6. 多设备写权限: 单写者相约 (云上一个 locks/daemon.lock + 心跳), 或按设备分片 notices/<deviceId>/, 读全部, 只写自己—后者更简单更稳。

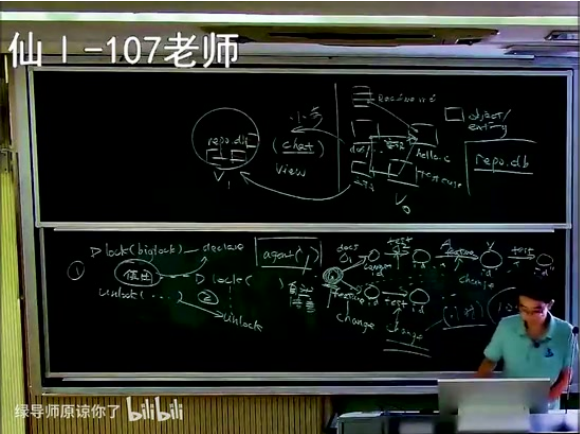
三种落地的难度

方案	改动	冲突处理
A. 挂载目录 (坚果云/iCloud/OneDrive 客户端 + pdbDir 指向它)	几乎零代码, 加个 fs.watch 触发 reindex	云客户端自己做冲突副本, 够用
B. WebDAV 适配器 (坚果云/Nextcloud)	src/cloud/ 一层, etag 尚可信	可做精确三方比对, 推荐
C. 网盘 API (百度/阿里/OneDrive Graph)	OAuth + 分片上传, 无稳定 etag	冲突检测难, 只适合「单向备份」而非双向同步

一句话建议

分三步走: 第一步只改 pdbDir + watcher (当天可用); 第二步抽 src/cloud/ 并改 archive 为一事一文件 (解决冲突这个真痛点); 第三步加 mail exporter。顺序很重要—先立「SQLite 不出本机 + 一事一文件」两条规矩, 否则网盘会让幂等设计从内部崩掉。

画面 01:38:00



画面 01:38:28